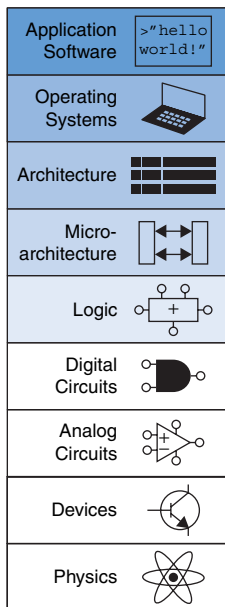


9.1 引言

输入/输出（I/O）系统用于将计算机与称为*peripherals*的外部设备连接。在个人计算机中，这些设备通常包括键盘、显示器、打印机和无线网络。在嵌入式系统中，设备可能包括烤面包机的加热元件、玩偶的语音合成器、发动机的燃油喷射器、卫星的太阳能板定位电机等。处理器通过地址和数据总线访问I/O设备，其方式与访问内存相同。

本章提供了I/O设备的具体示例。第9.2节展示了将I/O设备连接到处理器并从程序访问它的基本原理。第9.3节在嵌入式系统的背景下考察了I/O，展示了如何使用SparkFun的RED-V RedBoard（该板搭载RISC-V微控制器）来访问板载外设，包括通用、串行和模拟I/O，以及定时器和脉宽调制（PWM）。第9.4节给出了与其他常见设备（如字符型LCD、VGA显示器、蓝牙无线电和电机）接口的示例。



9.2 内存映射I/O

回顾第6.5.1节，地址空间的一部分是专用于I/O设备而非内存的。例如，假设物理地址范围0x20000000至0x20FFFFFF用于I/O。每个I/O设备在此范围内被分配一个或多个内存地址。对指定地址的存储操作会将数据发送到设备，而加载操作则从设备接收数据。这种与I/O设备通信的方法称为*memory-mapped I/O*。

在采用内存映射I/O的系统中，加载或存储操作可能访问内存或I/O设备。图e9.1展示了支持两个内存映射I/O设备所需的硬件。地址解码器决定哪个设备与处理器通信，它利用*Address*和*Mem Write*信号为其余部分生成控制信号。

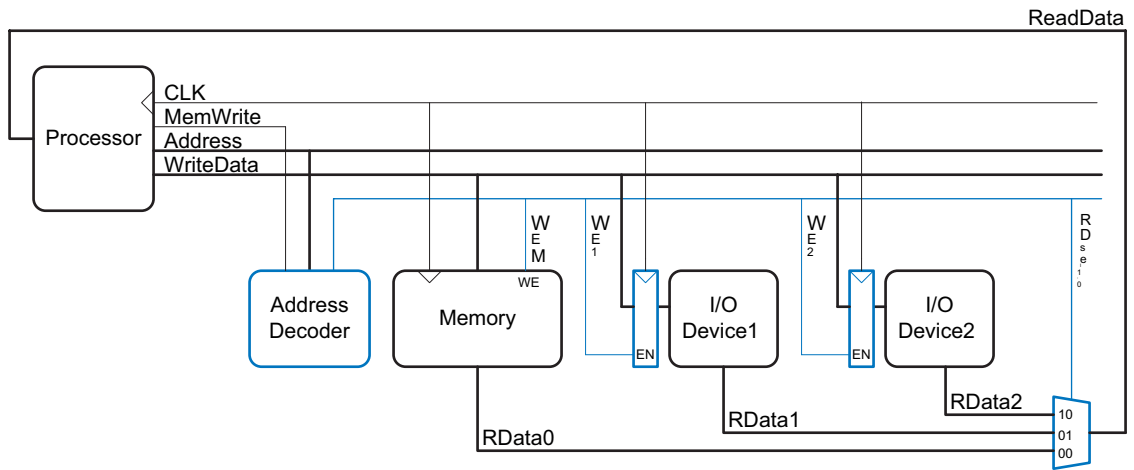


图 e9.1 内存映射I/O的支撑硬件

硬件。ReadData多路复用器在内存和各种I/O设备之间进行选择。写使能寄存器保存写入I/O设备的值。

示例 e9.1 与I/O设备通信

假设图 e9.1 中的 I/O 设备 1 被分配了内存地址 0x20001000。请给出将值 7 写入 I/O 设备 1 以及从 I/O 设备 1 读取输出值的 RISC-V 汇编代码。

解决方案 以下RISC-V汇编代码将值7写入I/O设备1。 .equ汇编指令将命名符号替换为给定值。因此， li s1, ioadr指令变为li s1, 0x20001000。

```
.equ ioadr 0x20001000

li s0, 7
li s1, ioadr
sw s0, 0(s1)
```

地址解码器检测到地址0x20001000和MemWrite=1，因此它断言WE1，即设备1寄存器的写使能信号。在下一个时钟沿， WriteData总线上的值7被写入该寄存器，其输出连接到I/O设备1的输入引脚。

为了从I/O设备1读取数据，处理器执行以下RISC-V汇编代码。

```
lw s0, 0(s1)
```

嵌入式处理器之所以得名，是因为它们通常嵌入在更大的系统（如玩具或汽车）中，且用户界面有限。相比之下，个人电脑中的处理器拥有键盘和屏幕等界面，便于用户编程或运行应用程序。但所有类型的处理器本质上相同——它们都执行指令。区别仅在于嵌入式处理器与传统处理器所使用的接口及外围设备不同。

地址解码器检测到地址0x20001000，因此将 $RDsel_{1:0}$ 设置为01。多路复用器随之选择 $RData1$ ，即来自设备1的读取数据，并将其连接到 $ReadData$ 总线，该总线的值随后被加载到处理器中的s0寄存器。

与I/O设备相关联的地址通常被称为*I/O registers*，因为它们可能对应于I/O设备中的物理寄存器，如图e9.1所示。

根据IC Insights的数据，2020年微控制器销量约为240亿颗，预计到2029年该市场将以每年10%的速度增长。独立微控制器的平均价格约为60美分，而8位微控制器集成在系统级芯片（SoC）上的成本不到十分之一美分。微控制器已变得无处不在且几乎不可见，2021年一辆普通新车中估计有100个或更多微控制器。

汽车是微控制器最大且增长最快的市场，其次是消费电子、工业系统、医疗设备和军事应用。2020年，16位微控制器占据了最大收入份额，但32位微控制器因其更强的性能，市场份额正在增长。

领先的微控制器制造商包括英飞凌、微芯科技、恩智浦、瑞萨、意法半导体和德州仪器。主流架构包括8051、AVR、PIC和ARM。ARM在90%移动设备的应用处理器领域近乎垄断。然而，RISC-V作为一种新兴的开源架构备受关注。

与I/O设备通信的软件称为*device driver*。您可能已经为打印机或其他I/O设备下载或安装了设备驱动程序。编写设备驱动程序需要详细了解I/O设备硬件，包括内存映射I/O寄存器的地址和行为。其他程序通过调用设备驱动程序中的函数来访问设备，而无需理解底层设备硬件。

9.3 嵌入式I/O系统

嵌入式系统使用处理器来控制与物理环境的交互。它们通常围绕微控制器单元（MCU）构建，这些单元将微处理器与一组易于使用的外设（如通用数字和模拟I/O引脚、串行端口、定时器等）结合在一起。微控制器通常价格低廉，旨在通过将大多数必要组件集成到单个芯片上来最小化系统成本和尺寸。大多数微控制器比一角硬币更小更轻，功耗仅为毫瓦级，成本从几角到几美元不等。微控制器根据其处理的数据大小进行分类。8位微控制器最小且最便宜，而32位微控制器提供更大的内存和更高的性能。

9.3.1 RED-V 板

为具体起见，本节将以实际系统为背景阐述嵌入式系统I/O。具体而言，我们将聚焦于SiFive公司的FE310-G002片上系统（SoC），该芯片包含一个320 MHz的32位RISC-V处理器，实现了RV32IMAC架构——即基础32位整数指令集（RV32I）加上乘法/除法（M）、原子内存访问（A）和压缩16位指令（C）扩展。这款MCU可用于SiFive的HiFive开发板，以及一系列第三方开发板，例如SparkFun的RED-V系列（提供Arduino和Thing Plus两种封装形式）。各小节描述的I/O接口之后将附有在FE310上运行的具体示例。所有示例均已在SparkFun的RED-V上测试通过。

RedBoard, 并且可以轻松在HiFive开发板上运行, 或适配到RED-V Thing Plus板上。

图 e9.2(a) 展示了 SparkFun 的 RED-V RedBoard, 其售价低于 40 美元, 尺寸为 2.7 英寸 × 2.1 英寸。该图还显示了每个引脚的信号名称, 我们将在本节中对其进行描述。该开发板可通过 5 V USB 电源供电, 或通过筒形插孔由 7 至 15 V 直流电源供电。板上的 FE310-G002 由板载的 3.3 V 和 1.8 V 稳压器供电。FE310-G002 具有 16 KiB 的 L1 指令缓存和 16 KiB 的数据 SRAM 暂存器。SparkFun 开发板还配有 32 MiB 的片外闪存, 可通过串行外设接口 (SPI) 访问, 用于存储程序和数据。

图e9.2(b)展示了RED-V Thing Plus, 其功能与RED-V RedBoard类似, 但采用更小的外形尺寸 (2.3英寸×0.9英寸), 可安装在面包板上以便于接口连接。其I/O引脚编号与RedBoard不同, 且在丝印层上难以辨认, 但在图e9.2(b)中已标注。

RED-V RedBoard 外形尺寸基于 Arduino R3 引脚布局设计, 旨在尽可能保持与众多该布局的 Arduino 扩展板的兼容性。所有 19 个可配置 I/O 信号均可通过排针访问, 工作电压为 3.3 V。排针还提供 3.3 V、5 V 和接地, 方便为连接到 RedBoard 的小型设备供电, 但 3.3 V 电源的最大总电流为 50 mA, 5 V 电源的最大总电流为 ~300 mA。

保持与 Arduino R3 引脚布局的兼容性导致每个引脚有多个名称: 丝印层 (印在板上的文字) 列出了标准的 Arduino 引脚编号, 但图e9.2中记录的 RED-V 引脚布局同时显示了 Arduino 引脚编号和对应的 FE310 GPIO (通用输入/输出) 引脚编号。对于

在本章中, RED-V RedBoard 简称为 *the RED-V*。

本书的配套材料 (见前言) 包含使用 RED-V 板的实验练习。

注意: 将 5 V 电压连接到 3.3 V 的 I/O 引脚之一可能会损坏该 I/O 引脚, 甚至可能损坏整个 FE310 芯片。若使用电压表探测 I/O 引脚, 请务必小心, 避免 VUSB 或 VBAT 意外接触到邻近引脚!

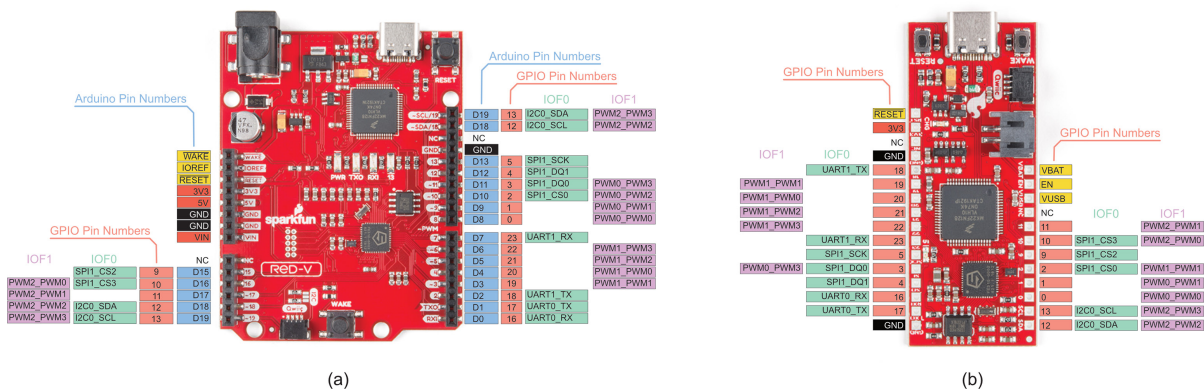


图 e9.2 (a) RED-V RedBoard; (b) RED-V Thing Plus 板
(照片由SparkFun提供, 遵循CC BY 2.0协议)

SiFive 由三位加州大学伯克利分校的研究人员于 2015 年创立：Krste Asanovic、Yunsup Lee 和 Andrew Waterman。SiFive 的愿景是让定制芯片开发比以往任何时候都更快、更经济。围绕开放的 RISC-V 指令集架构 (ISA)，他们开发了一个支持定制芯片系统级设计的平台。更多信息，包括 FE310-G002 数据手册，可在 sifive.com 上找到。

SiFive 的 RISC-V 微控制器持续演进。当您阅读本文时，可能已有配备更先进处理器和不同嵌入式 I/O 接口的新型号问世。不过，本文讨论的相同原理同样适用于该微控制器及其他微控制器。您将能找到相同类型的 I/O 接口和外设。您需要查阅数据手册，了解外设、芯片引脚、板级引脚之间的映射关系，以及每个外设对应的内存映射 I/O 地址（寄存器）。但正如本文所述，您仍需通过写入配置寄存器来初始化外设，并通过读写数据寄存器与外设进行通信。

例如，如图 e9.2 所示，GPIO5（FE310 引脚 5）对应 D13（Arduino 引脚 13）。RED-V Thing Plus 板虽未采用兼容 Arduino 的引脚命名方式，但也因此避免了单个引脚对应多个编号的问题。另请注意，在图 e9.2 中，部分 GPIO 引脚具有多重功能。例如，GPIO18（D2）还可作为 UART 1 的发送线（UART1_TX），具体将在后文说明。

在两块开发板上，GPIO5 均连接至蓝色 LED。该引脚在 RedBoard 上标记为 13（即 D13），在 Thing Plus 板上标记为 5。

本节首先介绍 FE310-G002 SoC，并描述用于内存映射 I/O 的通用设备驱动程序。本节其余部分将说明嵌入式系统如何执行通用数字、模拟和串行 I/O。

9.3.2 FE310-G002 片上系统

FE310-G002 SoC 是一款由 SiFive 设计的功能强大且价格低廉的微控制器芯片。它包含一个 RISC-V 微处理器，具有与第 7 章所述类似的 5 级流水线，以及众多 I/O 外设。FE310 采用 48 引脚、四方扁平无引脚封装。SiFive 发布了一份数据手册，描述了其许多特性和 I/O 寄存器；本章仅讨论其中一部分特性。

表 e9.1 展示了 FE310 的内存映射。启动时，处理器从外部闪存地址 0x20000000 开始执行代码。内存映射为外部闪存预留了最多 512 MiB 的空间，尽管当前的 RED-V 板卡容量要小得多：RED-V RedBoard 有 32 MiB 外部闪存，而 RED-V Thing Plus 有 4 MiB。该芯片还拥有 16 KiB 的 RAM，称为 *data tightly integrated memory* (DTIM)，位于地址 0x8000000。该 RAM 的加载延迟为 2 个周期，用于存储变量。各种外设通过内存映射分布在地址 0x02000000 到 0x1FFFFFFF 之间，将在后续章节中详细描述。这些外设包括通用输入输出、三个用于生成输出波形的脉宽调制 (PWM) 模块，以及多个用于连接外部设备的串行端口，包括三个串行外设接口 (SPI)、两个通用异步收发器 (UART) 和一个集成电路互连 (I²C) 接口。

图 e9.3 展示了 RED-V RedBoard 的简化示意图。该板通过 USB 电源接收 5 V 供电，稳压器产生 3.3 V 和 1.8 V 电压，用于 I/O、为低功耗常开核心及杂项功能供电。

9.3.3 通用数字 I/O

通用输入输出 (GPIO) 引脚用于读取或写入数字信号。至少，GPIO 引脚需要内存映射的 I/O 寄存器

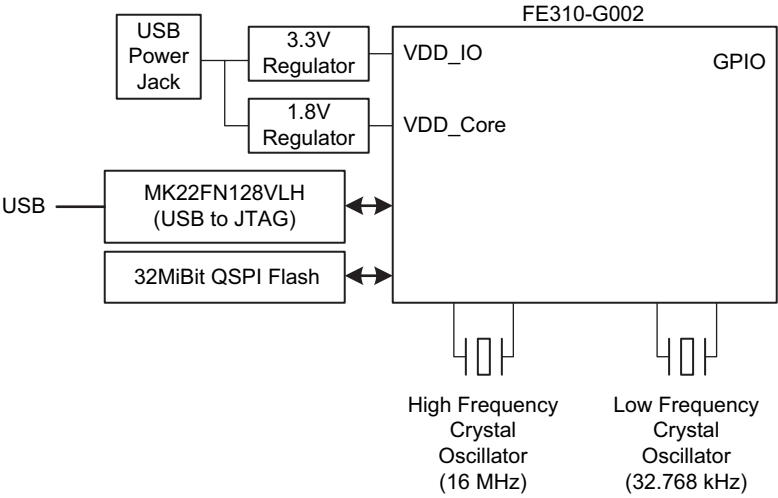
表 e9.1 FE310 内存映射

Base	Top	Attr.	Description	Notes
0x0000_0000	0x0000_0FFF	RWX A	Debug	Debug Address Space
0x0000_1000	0x0000_1FFF	R XC	Mode Select	On-Chip Non Volatile Mem- ory
0x0000_2000	0x0000_2FFF		Reserved	
0x0000_3000	0x0000_3FFF	RWX A	Error Device	
0x0000_4000	0x0000_FFFF		Reserved	
0x0001_0000	0x0001_1FFF	R XC	Mask ROM (8 KiB)	
0x0001_2000	0x0001_FFFF		Reserved	
0x0002_0000	0x0002_1FFF	R XC	OTP Memory Region	
0x0002_2000	0x001F_FFFF		Reserved	
0x0200_0000	0x0200_FFFF	RW A	CLINT	On-Chip Peripherals
0x0201_0000	0x07FF_FFFF		Reserved	
0x0800_0000	0x0800_1FFF	RWX A	E31 ITIM (8 KiB)	
0x0800_2000	0x0BFF_FFFF		Reserved	
0x0C00_0000	0x0FFF_FFFF	RW A	PLIC	
0x1000_0000	0x1000_0FFF	RW A	AON	
0x1000_1000	0x1000_7FFF		Reserved	
0x1000_8000	0x1000_8FFF	RW A	PRCI	
0x1000_9000	0x1000_FFFF		Reserved	
0x1001_0000	0x1001_0FFF	RW A	OTP Control	
0x1001_1000	0x1001_1FFF		Reserved	
0x1001_2000	0x1001_2FFF	RW A	GPIO	
0x1001_3000	0x1001_3FFF	RW A	UART 0	
0x1001_4000	0x1001_4FFF	RW A	QSPI 0	
0x1001_5000	0x1001_5FFF	RW A	PWM 0	
0x1001_6000	0x1001_6FFF	RW A	I2C 0	
0x1001_7000	0x1002_2FFF		Reserved	
0x1002_3000	0x1002_3FFF	RW A	UART 1	
0x1002_4000	0x1002_4FFF	RW A	SPI 1	
0x1002_5000	0x1002_5FFF	RW A	PWM 1	
0x1002_6000	0x1003_3FFF		Reserved	
0x1003_4000	0x1003_4FFF	RW A	SPI 2	
0x1003_5000	0x1003_5FFF	RW A	PWM 2	
0x1003_6000	0x1FFF_FFFF		Reserved	
0x2000_0000	0x3FFF_FFFF	R XC	QSPI 0 Flash (512 MiB)	Off-Chip Non-Volatile Mem- ory
0x4000_0000	0x7FFF_FFFF		Reserved	On-Chip Volatile Memory
0x8000_0000	0x8000_3FFF	RWX A	E31 DTIM (16 KiB)	
0x8000_4000	0xFFFF_FFFF		Reserved	

内存属性: R: 读取, W: 写入, X: 执行, C: 可缓存, A: 原子操作。经许可转载自 FE310-G0002 Manual, © 2019 SiFive, Inc. 表4。

读取输入引脚值、写入输出引脚值以及设置引脚方向。在许多嵌入式系统中，GPIO引脚可以与一个或多个专用外设共享，因此需要额外的配置寄存器来确定该引脚是通用用途还是专用用途。此外，当输入引脚上发生上升沿或下降沿等事件时，处理器可能会产生中断，并且配置寄存器可用于指定触发条件。

图 e9.3 RED-V 板原理图



一个中断。回顾一下，FE310有19个GPIO引脚。本节将从控制这些引脚的基本示例开始，然后探讨这些引脚的一些特殊用途。

图e9.4展示了三个发光二极管（LED）和三个开关连接到六个GPIO引脚。LED被设计为当驱动为1时发光，驱动为0时熄灭。限流电阻（通常约为300 Ω）与LED串联，用于设置亮度并避免GPIO电流能力过载。开关被设计为闭合时产生1，断开时产生0。如图所示，1 kΩ下拉电阻在开关断开时将引脚拉低至0。图e9.4标明了（Arduino）板上的引脚编号以及GPIO引脚编号。

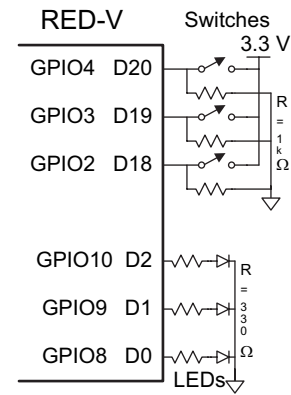


图 e9.4 GPIO 示例

表 e9.2 列出了 GPIO 寄存器及其相对于 GPIO 基地址 0x10012000 的地址偏移量，如 *FE310-G002 Manual* 的表 51 所示。我们首先关注顶部的四个寄存器（即内存映射 I/O 地址）。每个 GPIO 引脚映射到寄存器的一个位。从 input_val（输入值）寄存器读取可读取 GPIO 引脚的值，而向 output_val（输出值）寄存器写入则可写入 GPIO 引脚。在读取或写入引脚之前，必须设置输入和输出使能寄存器（input_en 和 output_en）以将引脚配置为输入或输出，并且必须清除硬件驱动功能使能寄存器（iof_en）以将引脚配置为 GPIO 控制。

GPIO 内存映射输入输出
我们通过编写一个程序来演示如何使用GPIO引脚，该程序读取开关状态并通过GPIO控制LED。

表 e9.2 GPIO 寄存器偏移量

Offset	Name	Description
0x00	input_val	Pin value
0x04	input_en	Pin input enable*
0x08	output_en	Pin output enable*
0x0C	output_val	Output value
0x10	pue	Internal pull-up enable*
0x14	ds	Pin drive strength
0x18	rise_ie	Rise interrupt enable
0x1C	rise_ip	Rise interrupt pending
0x20	fall_ie	Fall interrupt enable
0x24	fall_ip	Fall interrupt pending
0x28	high_ie	High interrupt enable
0x2C	high_ip	High interrupt pending
0x30	low_ie	Low interrupt enable
0x34	low_ip	Low interrupt pending
0x38	iof_en	HW-driven functions enable
0x3C	iof_sel	HW-driven functions selection
0x40	out_xor	Output XOR (invert)

标有*的寄存器在启动时异步复位为0，以确保GPIO引脚处于非活动状态。经许可转载自SiFive FE310-G0002 Manual, © 2019 SiFive, Inc. 的表52。

与GPIO引脚交互的五个最重要的寄存器，如上所述，分别是input_val、input_en、output_en、output_val和iof_en，其偏移地址相对于基地址分别为0x0、0x4、0x8、0xC和0x38。每个寄存器宽度为32位，最多可控制32个GPIO，但该芯片上实际只有19个GPIO物理存在。

要读取GPIO n，程序需设置input_en（输入使能）寄存器的第n位，然后读取input_val（输入值）寄存器并查看第n位。类似地，要驱动GPIO n，程序需设置output_en（输出使能）寄存器的第n位，然后将所需值写入output_val（输出值）寄存器的第n位。在这两种情况下，第n位

在位操作中，“置位”表示将某位写为1，“清零”表示将某位写为0。

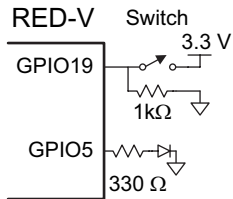


图 e9.5 GPIO 引脚 5 上的 LED 输出与 GPIO 引脚 19 上的开关输入

必须清除 `iof_en` 寄存器，以确保引脚由 GPIO 控制器驱动，而非芯片上的其他硬件。

代码示例e9.1展示了一个简单程序，该程序读取连接到GPIO19的开关值，并据此打开或关闭连接到GPIO5的板载LED。硬件设置如图e9.5所示。为了访问内存映射I/O，程序首先声明了指向上述地址中五个寄存器的指针。每个指针的类型为`uint32_t*`，因为这些寄存器包含无符号32位值。程序向`input_en`寄存器的第19位写入1，并向`output_en`寄存器的第5位写入1，以将GPIO引脚19配置为输入，GPIO引脚5配置为输出。注意我们如何使用移位操作(`1 << 19`)在第19位设置1，并将其与使能寄存器的现有内容进行或运算，以开启该位而不影响可能已开启的其他位。然后，我们在`iof_en`寄存器的第5位和第19位写入0，以确保引脚由GPIO控制器驱动。要向某位写入0，我们将`iof_en`与除该位外所有位置均为1的值进行与运算，从而强制该位为低电平，同时不影响其他位。接下来，程序反复读取输入引脚并写入输出引脚。为读取输入引脚，程序读取`input_val`寄存器，将值右移19位（将引脚19的值移至第0位），然后与0x1进行按位与运算以仅保留第0位，得到一个与原始第19位值对应的0或1。要向`output_val`寄存器的某位写入高电平，我们使用或运算，如同在使能寄存器中开启某位一样。要向`output_val`寄存器的某位写入0，我们采用与上述清除`iof_en`寄存器中位相同的方法。

代码示例 e9.1 根据开关输入设置 GPIO 输出

```
#include <stdint.h>
int main(void) {
    volatile uint32_t *input_val  = (uint32_t*)0x10012000;
    volatile uint32_t *input_en   = (uint32_t*)0x10012004;
    volatile uint32_t *output_en  = (uint32_t*)0x10012008;
    volatile uint32_t *output_val = (uint32_t*)0x1001200C;
    volatile uint32_t *iof_en     = (uint32_t*)0x10012038;
    int val;

    *iof_en    &= ~(1 << 19);           // Pin 19 is a GPIO
    *input_en  |= (1 << 19);            // Pin 19 is an input
    *iof_en    &= ~(1 << 5);            // Pin 5 is a GPIO
    *output_en |= (1 << 5);              // Pin 5 is an output
    while(1) {
        val = (*input_val >> 19) & 1;    // Read value on pin 19
        if (val) *output_val |= (1 << 5); // Turn ON pin 5
        else    *output_val &= ~(1 << 5); // TURN OFF pin 5
    }
}
```

其他GPIO寄存器

表e9.2列出了其他几个值得关注的GPIO控制寄存器，特别是引脚驱动强度（ds）、内部上拉使能（pue）以及I/O功能（iof_sel和iof_en）寄存器。

ds 寄存器控制每个引脚的最大输出电流。默认值（0）将 I_{OL}/I_{OH} 配置为 15 至 16 mA，而将某个引脚的 ds 设置为 1 会将该引脚的输出电流适度提升至 21 mA，这可能有助于驱动更亮的 LED。

pue寄存器配置了一个内部上拉电阻。图e9.4展示了一个外部 *pull-down* 电阻的示例。如果开关上的电源和地线连接被反转，那么该电阻将变为一个 *pull-up* 电阻，在开关未连接时将引脚驱动为1。在这种情况下，当开关被按下时，引脚将降至0。为了节省成本和电路板空间，许多微控制器包含内部上拉电阻，可以通过软件选择启用。将pue寄存器的某一位写为1，即可激活对应GPIO引脚的内部上拉电阻。根据FE310-G002数据手册的表4.2，当引脚电压为0V时，上拉电流为85μA。因此，有效上拉电阻为 $3.3V/85\mu A = 39k\Omega$ ($V/I=R$)。

如表e9.3所示，大多数GPIO引脚还可执行特殊功能，例如作为串行端口或脉宽调制（PWM）输出。我们将在本章后续部分详细讨论这些功能。iof_sel和iof_en寄存器共同决定每个引脚是作为GPIO还是特殊功能使用。当iof_en为0（默认值）时，该引脚作为GPIO使用；当其为1时，则启用特殊功能。特殊功能根据该引脚的iof_sel位从表e9.3中选择。例如，要使用GPIO11生成脉宽调制波形，需将iof_sel和iof_en的第11位设置为1，然后使用PWM寄存器控制输出。iof_en映射到地址0x10012038，iof_sel映射到0x1001203C。表e9.3列出了32个GPIO，但请注意RED-V板仅包含19个GPIO：GPIO 0至5、9至13以及16至23。

9.3.4 设备驱动程序

正如我们在代码示例 e9.1 中所见，程序员可以通过读写内存映射 I/O 寄存器来直接操作 I/O 设备。然而，更好的编程实践是调用访问内存映射 I/O 的函数。这些函数被称为 *device drivers*。使用设备驱动程序的一些好处包括：

当代码涉及一个命名清晰的函数调用，而不是向晦涩内存地址的位域写入时，代码更易于阅读。► 熟悉I/O设备深层工作原理的人可以编写设备驱动程序，而普通用户无需理解细节即可调用它。

表 e9.3 GPIO引脚特殊功能映射

GPIO Number	IOF0	IOF1
0		PWM0_PWM0
1		PWM0_PWM1
2	SPI1_CS0	PWM0_PWM2
3	SPI1_DQ0	PWM0_PWM3
4	SPI1_DQ1	
5	SPI1_SCK	
6	SPI1_DQ2	
7	SPI1_DQ3	
8	SPI1_CS1	
9	SPI1_CS2	
10	SPI1_CS3	PWM2_PWM0
11		PWM2_PWM1
12	I2C0_SDA	PWM2_PWM2
13	I2C0_SCL	PWM2_PWM3
14		
15		
16	UART0_RX	
17	UART0_TX	
18	UART1_TX	
19		PWM1_PWM1
20		PWM1_PWM0
21		PWM1_PWM2
22		PWM1_PWM3
23	UART1_RX	
24		
25		
26	SPI2_CS0	
27	SPI2_DQ0	
28	SPI2_DQ1	
29	SPI2_SCK	
30	SPI2_DQ2	
31	SPI2_DQ3	

经许可转载自SiFive *FE310-G0002 Manual* 的表53
, © 2019 SiFive, Inc.

代码更容易移植到具有不同内存映射或I/O设备的其他处理器上，因为只需更改设备驱动程序即可。►如果设备驱动程序是操作系统（OS）的一部分，操作系统可以控制对系统中多个程序共享的物理设备的访问，并管理安全性（例如，防止恶意程序在您向网页浏览器输入密码时读取键盘输入）。

本章将开发一个名为EasyREDVIO的简单设备驱动程序，用于访问FE310外设，以便您了解设备驱动程序的底层工作原理。要访问FE310的所有功能，用户可能更倾向于使用Freedom Metal环境，该环境提供了便捷的软件接口，用于控制SiFive Core IP功能及外设设备。Freedom Metal功能强大，因为其API的编写方式使其能够在任何拥有Freedom Metal板级支持包（BSP）的设备上运行。BSP是一个包含驱动程序和其他常用例程的软件包。SiFive还提供了Freedom E软件开发工具包（SDK）和Freedom Studio，使用户能够为任何SiFive核心开发软件。

EasyREDVIO 及本章中的代码示例可从教材网站下载（参见前言）。有关 Freedom Metal 的更多信息和文档，请访问 <https://github.com/sifive/freedom-metal>。

示例 e9.2 C语言中的设备驱动程序

访问和修改内存映射I/O的值是通过读取或写入内存地址来实现的。在汇编中，这通过`lw`和`sw`指令完成。如代码示例e9.2所示，C语言可以使用指针实现相同的功能，但为每个内存映射I/O寄存器声明指针既繁琐又容易出错。在C语言中，更自然的描述和控制内存映射I/O的方式是使用结构体。

如附录C.8.5节所述，C语言中的结构体是一种将多种不同数据类型组合为单一单元的方式。在内存映射寄存器的场景下使用结构体，可以通过指定寄存器或字段的名称（而非内存地址）与I/O设备进行通信。C程序可为内存映射外设声明一个结构体，按内存映射中出现的顺序列出寄存器，随后声明指向该结构体的指针，并通过该结构体指针访问外设。

通过编写pinMode、digitalRead和digitalWrite函数来配置引脚方向并进行读取或写入，从而启动EasyREDVIO库。

pinMode函数接受两个输入：引脚编号和模式。例如，pinMode(5, INPUT)将GPIO引脚5设置为输入，pinMode(17, OUTPUT)将GPIO引脚17设置为输出。digitalRead接受一个输入，即引脚编号，并返回该引脚的值。例如，digitalRead(19)读取GPIO19的值。digitalWrite接受两个输入：引脚编号和值。例如，digitalWrite(3, 1)向GPIO引脚3写入1，digitalWrite(5, 0)向GPIO引脚5写入0。

编写这些函数后，编写一个C程序，利用这些函数读取三个开关的状态，并根据图e9.4中的硬件点亮相应的LED。

解决方案 EasyREDVIO 代码如下。函数必须选择要访问的寄存器以及这些寄存器中的位。例如，要

将引脚配置为输入时，pinMode 必须将该引脚在 input_en 中的对应位置位，并清除该引脚在 output_en 中的对应位。digitalWrite 通过写入 output_val 来处理输出 1 或 0。digitalRead 读取 input_val 中所需的位。

GPIO 结构体（struct）按名称指定了 32 位寄存器。随后两条 define 语句指定了 GPIO 的基地址（GPIO0_BASE），并在该基地址处实例化了一个 GPIO 类型的指针。结构体中的每个 32 位变量随后从该基地址开始按升序在内存中定位。

代码示例 e9.2 开关与LED的GPIO

```
// EasyREDVIO.h
// Joshua Brake, David Harris, and Sarah Harris, 7 October 2020

#include <stdint.h>

#define INPUT 0
#define OUTPUT 1

// Define statements to map Arduino pin names to FE310 GPIO pin number according to Figure e9.2
#define D0 16
#define D1 17
#define D2 18
#define D3 19
#define D4 20
#define D5 21
#define D6 22
#define D7 23
#define D8 0
#define D9 1
#define D10 2
#define D11 3
#define D12 4
#define D13 5
#define D15 9
#define D16 10
#define D17 11
#define D18 12
#define D19 13

// Declare a GPIO structure defining the GPIO registers in the order they appear in Table e9.2
typedef struct {
    volatile uint32_t input_val; // (GPIO offset 0x00) Pin value
    volatile uint32_t input_en; // (GPIO offset 0x04) Pin input enable*
    volatile uint32_t output_en; // (GPIO offset 0x08) Pin output enable*
    volatile uint32_t output_val; // (GPIO offset 0x0C) Output value
    volatile uint32_t pue; // (GPIO offset 0x10) Internal pull-up enable*
    volatile uint32_t ds; // (GPIO offset 0x14) Pin drive strength
    volatile uint32_t rise_ie; // (GPIO offset 0x18) Rise interrupt enable
    volatile uint32_t rise_ip; // (GPIO offset 0x1C) Rise interrupt pending
    volatile uint32_t fall_ie; // (GPIO offset 0x20) Fall interrupt enable
    volatile uint32_t fall_ip; // (GPIO offset 0x24) Fall interrupt pending
    volatile uint32_t high_ie; // (GPIO offset 0x28) High interrupt enable
    volatile uint32_t high_ip; // (GPIO offset 0x2C) High interrupt pending
    volatile uint32_t low_ie; // (GPIO offset 0x30) Low interrupt enable
    volatile uint32_t low_ip; // (GPIO offset 0x34) Low interrupt pending
    volatile uint32_t iof_en; // (GPIO offset 0x38) HW-Driven functions enable
    volatile uint32_t iof_sel; // (GPIO offset 0x3C) HW-Driven functions selection
    volatile uint32_t out_xor; // (GPIO offset 0x40) Output XOR (invert)
    // Registers marked with * are asynchronously reset to 0 at startup
} GPIO;
```

```

// Define the base address of the GPIO registers (see Table e9.1) and a pointer to this
// structure
// The 0x..U notation in 0x10012000U indicates an unsigned hexadecimal number
#define GPIO0_BASE (0x10012000U)
#define GPIO0 ((GPIO*) GPIO0_BASE)

// To access the members of the structure, the member-access operator -> is used.

void pinMode(int gpio_pin, int function) {
    switch(function) {
        case INPUT:
            GPIO0->input_en |= (1 << gpio_pin); // Sets a pin as an input
            GPIO0->output_en &= ~(1 << gpio_pin); // Clear output_en bit
            GPIO0->iof_en &= ~(1 << gpio_pin); // Disable IOF
            break;
        case OUTPUT:
            GPIO0->output_en |= (1 << gpio_pin); // Set pin as an output
            GPIO0->input_en &= ~(1 << gpio_pin); // Clear input_en bit
            GPIO0->iof_en &= ~(1 << gpio_pin); // Disable IOF
            break;
    }
}

void digitalWrite(int gpio_pin, int val) {
    if (val) GPIO0->output_val |= (1 << gpio_pin);
    else    GPIO0->output_val &= ~(1 << gpio_pin);
}

int digitalRead(int gpio_pin) {
    return (GPIO0->input_val >> gpio_pin) & 0x1;
}

// The program below reads switches and writes LEDs. It sets pins 2 to 4 as inputs (for the
// switches) and pins 7 to 9 as outputs (for the LEDs). It then continuously reads the
// switches and writes their values to the corresponding LEDs.

#include "EasyREDVIO.h"
int main(void) {
    // Set GPIO 4:2 as inputs
    pinMode(2, INPUT);
    pinMode(3, INPUT);
    pinMode(4, INPUT);
    // Set GPIO 10:8 as outputs
    pinMode(8, OUTPUT);
    pinMode(9, OUTPUT);
    pinMode(10, OUTPUT);
    while (1) { // Read each switch and write corresponding LED
        digitalWrite(8, digitalRead(2));
        digitalWrite(9, digitalRead(3));
        digitalWrite(10, digitalRead(4));
    }
}

```

9.3.5 串行I/O

微控制器可以通过使用多根线向外围设备发送多个比特，或者通过单根线串行发送多个比特。前者称为*parallel I/O*，后者称为*serial I/O*。串行I/O很流行，尤其是在引脚有限的情况下，因为它使用的线少，且对许多应用而言速度足够快。事实上，它如此流行，以至于已经建立了许多串行I/O标准。

微控制器提供专用硬件，可轻松通过这些标准发送数据。本节介绍SPI和UART标准串行接口。

其他常见的串行标准包括集成电路间总线（I²C）、通用串行总线（USB）和以太网。I²C（读作“*I平方C*”）是一种具有时钟和双向数据引脚的2线接口，其使用方式与SPI类似。USB和以太网是更复杂的高性能标准。FE310通过板载专用外设支持SPI、UART和I²C。

术语*master/slave*曾经很常见（代替*controller/peripheral*），但现在已过时。现在使用*Serial Data Out (SDO)*代替*master-*，而*Serial Data In (SDI)*或*controller-in peripheral-out (CIPO)*

串行外设接口（SPI）
SPI（读作“*S-P-I*”）是一种简单同步串行协议，易于使用且速度相对较快。其物理接口包含三个引脚：串行时钟（SCK）、串行数据输出（SDO）和串行数据输入（SDI）。SPI将*controller*设备与*peripheral*设备连接，如图e9.6(a)所示。控制器产生时钟，并通过在SCK上发送时钟脉冲来启动通信。控制器每个周期从其SDO引脚向外设的SDI引脚发送一位数据，从最高有效位开始。外设可同时从其SDO引脚向控制器的SDI引脚回复数据。图e9.6(b)展示了8位数据传输的SPI波形。数据在SCK下降沿变化，并在上升沿稳定以供采样。SPI接口还可发送低电平有效的片选信号，以通知接收器数据即将到来。

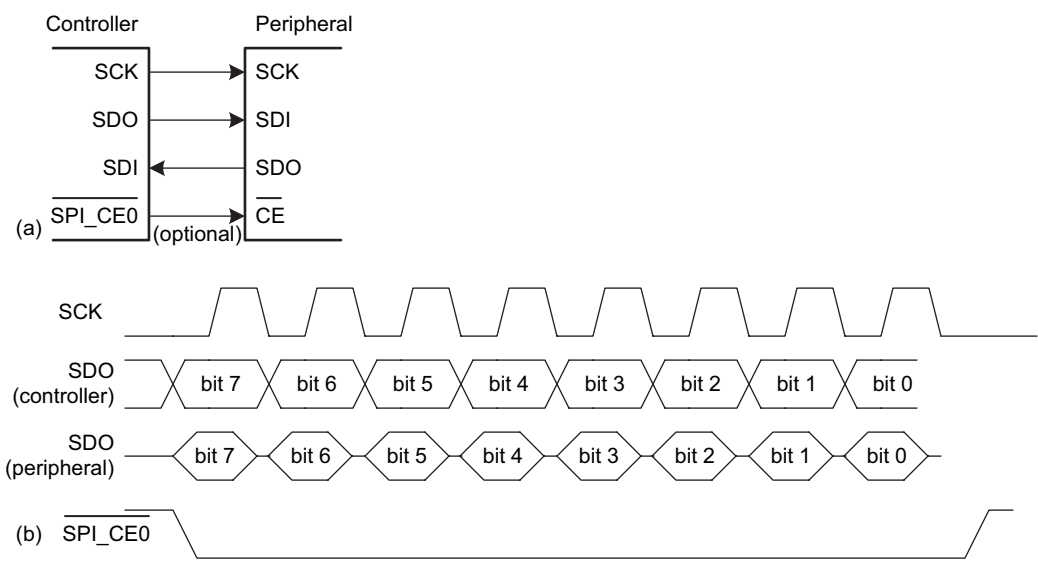


图 e9.6 SPI 共 配置: (a) SPI控制器-外设连接图, (b) 示例S PI数据信号

FE310具有三个SPI控制器端口，但只有两个（SPI1和SPI2）可供用户使用。剩余的SPI控制器端口SPI0用于与外部闪存通信，以存储程序和数据。本节描述SPI1控制器端口，该端口可通过GPIO引脚5:2访问。SPI2控制器端口与之相同，区别在于它连接到不同的GPIO引脚，且其控制寄存器位于不同的内存地址。要将引脚用于SPI而非GPIO，需将其iof_sel位设置为0以选择SPI1功能，并将iof_en位设置为1以使SPI控制器能够访问这些引脚。当FE310向SPI txdata寄存器写入数据时，数据会串行传输至外设。同时，从外设接收的数据会被收集，传输完成后FE310可从rxdata读取该数据。

微控制器上的SPI端口通常提供多种配置选项，以满足外设设备的需求。在设计与特定外设设备通信的接口时，必须正确配置控制器，以确保通过链路传输的数据能被正确解析。

两个常见的配置参数是时钟极性（CPOL）和时钟相位（CPHA）。CPOL设置时钟空闲时的电平，CPHA设置数据（SDO和SDI）被采样（和改变）时的时钟边沿。如果CPOL = 1，则在不传输数据时SCK保持高电平（1）；如果CPOL = 0，则空闲时SCK保持低电平（0）。如果CPHA = 0，则在SCK的上升沿采样数据（并在下降沿改变数据）；如果CPHA = 1，则在SCK的下降沿采样数据（并在上升沿改变数据）。数据发生变化的边沿也被称为*shifting edge*，因为底层硬件通常是一个移位寄存器。图e9.7展示了CPHA和CPOL的四种可能组合。图 e9.6中的示例显示CPOL = 0且CPHA = 0。

术语*port*指一个引脚或一组相关引脚。

SPI 每次传输时总是双向发送数据。若系统仅需单向通信，可忽略不需要的数据。例如，若控制器只需向外设发送数据，则可忽略从外设接收的字节。若控制器仅需从外设接收数据，则仍需通过发送一个外设会忽略的任意字节来触发 SPI 通信，随后即可读取从外设接收的数据。SPI 时钟（SCK）仅在控制器传输数据时发生跳变。

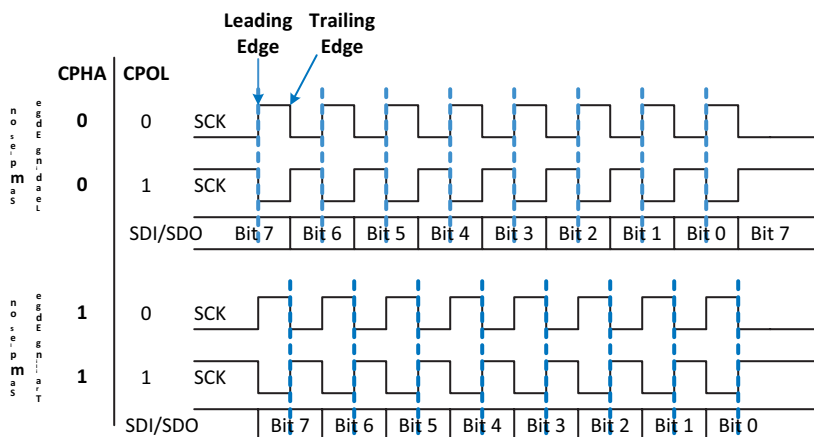


图 e9.7 SPI外设的时序图与配置

表 e9.4 SPI 寄存器内存映射

Offset	Name	Description
0x00	sckdiv	Serial clock divisor
0x04	sckmode	Serial clock mode
0x08	Reserved	
0x0C	Reserved	
0x10	csid	Chip select ID
0x14	csdef	Chip select default
0x18	csmode	Chip select mode
0x1C	Reserved	
0x20	Reserved	
0x24	Reserved	
0x28	delay0	Delay control 0
0x2C	delay1	Delay control 1
0x30	Reserved	
0x34	Reserved	
0x38	Reserved	
0x3C	Reserved	
0x40	fmt	Frame format
0x44	Reserved	
0x48	txdata	Tx FIFO Data
0x4C	rxdata	Rx FIFO data
0x50	txmark	Tx FIFO watermark
0x54	rxmark	Rx FIFO watermark
0x58	Reserved	
0x5C	Reserved	
0x60	fctrl	SPI flash interface control*
0x64	ffmt	SPI flash instruction format*
0x68	Reserved	
0x6C	Reserved	
0x70	ie	SPI interrupt enable
0x74	ip	SPI interrupt pending

经许可转载自SiFive *FE310-G0002 Manual*, © 2019 Si Five, Inc. 表65。

表 e9.4 显示了与 SPI1 相关的控制寄存器，表 e9.5 显示了关键寄存器的字段。sckdiv（见表 e9.4）通过为选定的输入外设时钟指定一个除数（div）来配置 SPI 时钟频率——在 RED-V 板上，外设时钟的默认频率为 16 MHz。SPI 时钟的频率由 $f_{sck} = \frac{f_{in}}{2(div + 1)}$ 给出。例如，如果 div = 15，则串行时钟

是 $f_{sck} = \frac{16\text{ MHz}}{2(15 + 1)} = 500\text{ kHz}$ 。如果频率过高（在面包板上超过 >~1 MHz，或在未端接的印刷电路板上超过几十 MHz），SPI 连接可能会因反射、串扰或其他信号完整性问题而变得不可靠。

表 e9.5 SPI 寄存器位域

Serial Clock Divisor Register (sckdiv)				
Register Offset		0x0		
Bits	Field Name	Attr.	Rst.	Description
[11:0]	div	RW	0x3	Divisor for serial clock. div_width bits wide.
[31:12]	Reserved			

Serial Clock Mode Register (sckmode)				
Register Offset		0x4		
Bits	Field Name	Attr.	Rst.	Description
0	pha	RW	0x0	Serial clock phase
1	pol	RW	0x0	Serial clock polarity
[31:2]	Reserved			

Transmit Data Register (txdata)				
Register Offset		0x48		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	data	RW	0x0	Transmit data
[30:8]	Reserved			
31	full	RO	X	FIFO full flag

Receive Data Register (rxdata)				
Register Offset		0x4C		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	data	RO	X	Received data
[30:8]	Reserved			
31	empty	RW	X	FIFO empty flag

配置寄存器有许多未使用或“保留”位。这些位可能在芯片的未来版本中使用，因此不应写入，以免在未来导致意外后果。

经许可转载自SiFive FE310-G0002 Manual, © 2019 SiFive, Inc.的表66、67、80和81。

sckmode 控制时钟的相位和极性。sckmode 仅使用最低两位。位0为CPHA，位1为CPOL。

txdata 用于通过 SPI 通道发送一个字节，读取 rxdata 可获取接收到的字节。只有写入 txdata 的最低有效字节 (LSB) 会被发送。FE310 上的 SPI 实例在发送和接收数据寄存器上均具有 8 项先进先出 (FIFO) 缓冲区。这意味着当数据写入 txdata 寄存器时，它会被放入 FIFO 缓冲区，并由 SPI 外设内部的硬件负责发送出去。txdata 寄存器的最高有效位 (msb) 是一个名为 full 的标志位，当 FIFO 已满且无法接收更多数据时，该位为 1。

从FE310 SPI rxdata寄存器读取数据时必须小心。SPI控制器的设计使得寄存器中的数据在读取时会从接收FIFO中移除。要检查rxdata寄存器是否包含有效数据，应读取该寄存器

一旦检查空位以确定数据是否有效，程序员应注意避免对每个字节多次读取寄存器，否则会导致数据丢失。

寄存器csid、csdef和csmode可选用于控制与片选线的控制和配置相关的参数。或者，片选引脚可以配置为GPIO输出引脚，并通过digitalWrite在软件中进行控制。

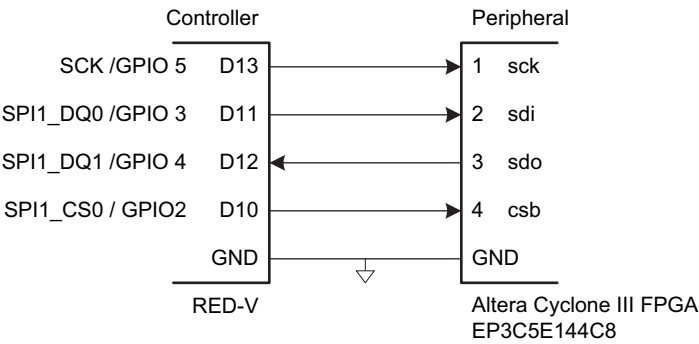
某些SPI寄存器将多个小信息字段打包到一个32位字中。在C语言中，我们可以通过冒号和数字声明每个字段的位数，作为位域结构的一部分。示例e9.3展示了如何使用位域和结构体来定义这些寄存器。

示例 e9.3 通过 SPI 发送和接收字节

设计一个系统，通过SPI在FE310控制器与FPGA外设之间进行通信。绘制接口原理图。编写FE310发送字符“A”并接收返回字符的C代码。编写FPGA上SPI外设的HDL代码。如果外设仅需接收数据，如何简化其设计？

解决方案 图e9.8展示了FE310控制器与FPGA外设之间通过SPI1的连接。引脚编号来自组件数据手册（例如，FE310的表e9.3）。请注意，图中同时显示了引脚编号和信号名称，以指示物理和逻辑连接关系。当SPI连接启用时，这些引脚不能用于GPIO。

图e9.8 RED-V到Altera Cyclone FPGA连接图



以下代码来自EasyREDVIO.h，用于初始化SPI通道以及发送和接收一个字符。该文件首先声明了SPI位域和内存映射。pinMode函数经过泛化，支持I/O功能以及输入和输出。spiSendReceive函数完成一次完整的SPI事务，发送并接收一个字节。它首先检查发送FIFO是否未满足且可以接受新条目。如果是，则将字符写入发送FIFO。

将被移出。传输完成后，读取rxdata寄存器。此处需特别注意，因为rxdata寄存器的空标志位在每次读取该寄存器时都会更新。因此，应读取完整的32位rxdata寄存器。随后，在确认空标志位未被置位（即数据有效）后，返回接收到的字节。

Code Example e9.3 SPI FUNCTIONS

```

/////////////////////////////////////////////////////////////////
// SPI Registers
/////////////////////////////////////////////////////////////////
typedef struct {
    volatile uint32_t    div          :   12; // Clock divisor
    volatile uint32_t    :           20;
} sckdiv_bits;

typedef struct {
    volatile uint32_t    pha          :    1; // Serial clock phase
    volatile uint32_t    pol          :    1; // Serial clock polarity
    volatile uint32_t    :           30;
} sckmode_bits;

...

typedef struct {
    volatile uint32_t    data          :    8; // Transmit data
    volatile uint32_t    :           23;
    volatile uint32_t    full          :    1; // FIFO full flag
} txdata_bits;

typedef struct {
    volatile uint32_t    data          :    8; // Received data
    volatile uint32_t    :           23;
    volatile uint32_t    empty         :    1; // FIFO empty flag
} rxdata_bits;

// Pin modes
#define INPUT 0
#define OUTPUT 1
#define GPIO_IOF0 2
#define GPIO_IOF1 3

void pinMode(int gpio_pin, int function) {
    switch(function) {
        case INPUT:
            GPIO0->input_en   |= (1 << gpio_pin); // Set a pin as an input
            GPIO0->iof_en      &= ~(1 << gpio_pin); // Disable IOF
            break;
        case OUTPUT:
            GPIO0->output_en  |= (1 << gpio_pin); // Set pin as an output
            GPIO0->iof_en      &= ~(1 << gpio_pin); // Disable IOF
            break;
        case GPIO_IOF0:
            GPIO0->iof_en      |= (1 << gpio_pin); // Enable IOF
            GPIO0->iof_sel     &= ~(1 << gpio_pin); // IO Function 0
            break;
        case GPIO_IOF1:
            GPIO0->iof_en      |= (1 << gpio_pin); // Enable IOF
            GPIO0->iof_sel     |= (1 << gpio_pin); // IO Function 1
            break;
    }
}

```

```

void spiInit(uint32_t clkdivide, uint32_t cpol, uint32_t cpha) {
    // Initially assigning SPI pins (GPIO 2-5) to HW I/O function 0
    pinMode(3, GPIO_IOP0); // SDO
    pinMode(4, GPIO_IOP0); // SDI
    pinMode(5, GPIO_IOP0); // SCK

    digitalWrite(2, 1); // make sure CS0 doesn't pulse low
    pinMode(2, OUTPUT); // CS0 is manually controlled

    SPI1->sckdiv.div = clkdivide; // Set the clock divisor

    SPI1->sckmode.pol = cpol; // Set the polarity
    SPI1->sckmode.pha = cpha; // Set the phase
}

/* Transmits a character (1 byte) over SPI and returns the received character.
 * send: the character to send over SPI
 * return value: the character received over SPI */
uint8_t spiSendReceive(uint8_t send) {
    while(SPI1->txdata.full); // Wait until transmit FIFO is ready for new data
    SPI1->txdata.data = send; // Transmit the character over SPI

    rxdata_bits rxdata;
    while (1) {
        rxdata = SPI1->rxdata; // Read the rxdata register EXACTLY once
        if (!rxdata.empty) { // If the empty bit was not set, return the data
            return (uint8_t)rxdata.data;
        }
    }
}

```

代码示例e9.4中的C代码初始化了SPI，然后发送并接收一个字符。使用公式 $f_{clk} = \frac{f_{in}}{2^{(div+1)}}$ ，其中 f_{in} 为16 MHz核心时钟，它将SPI时钟设置为500 kHz。

Code Example e9.4 SPI FUNCTIONS

```

#include "EasyREDVIO.h"

int main(void) {
    uint8_t volatile received;

    // Initialize the SPI
    // Clock divisor of div = 15, CPOL = 0, CPHA = 0
    spiInit(15, 0, 0);

    digitalWrite(2, 0); // enable the peripheral (chip select = 0), if necessary
    received = spiSendReceive('A'); // Send letter A and receive byte
    digitalWrite(2, 1); // turn off chip enable
}

```

如果外设仅需从控制器接收数据，则其为一个简单的移位寄存器，如HDL示例e9.1所示。在每个sck上升沿，一个新的sdi位会移入移位寄存器，从数据的最高有效位开始。经过八个时钟周期后，整个字节已被读入移位寄存器。

HDL Example e9.1 HDL FOR SPI PERIPHERAL (RECEIVER ONLY)

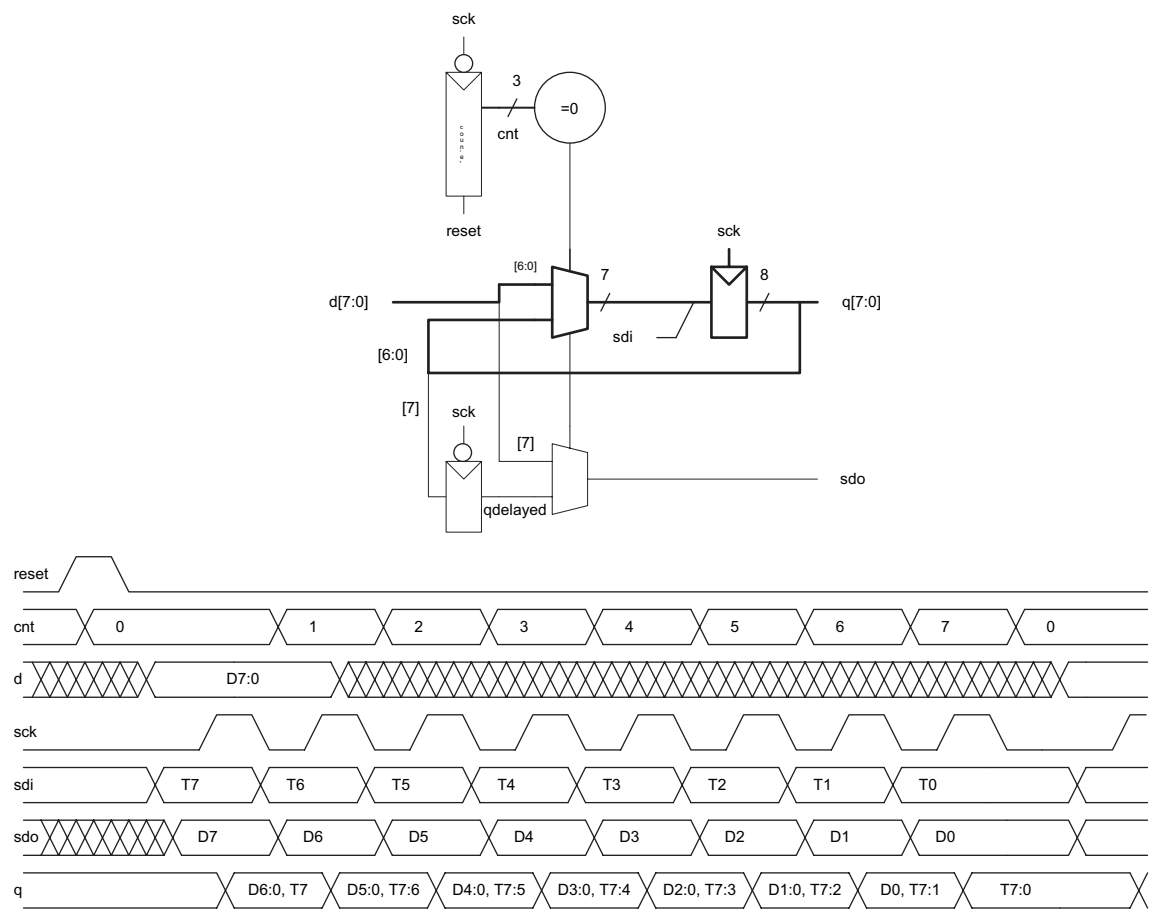
```
module spi_peripheral_receive_only(input logic sck, // From controller
                                  input logic sdi, // From controller
                                  output logic [7:0] q); // Data received

    always_ff @(posedge sck)
        q <= {q[6:0], sdi}; // shift register
endmodule
```

HDL示例e9.2给出了能够同时发送和接收数据的SPI外设（即SPI收发器）的SystemVerilog代码，图e9.9展示了其框图以及在CPHA = CPOL = 0下的时序。主要组件仍然是一个移位寄存器，如图e9.9右侧所示。该移位寄存器将待发送的字节(d[7:0])并行加载到移位寄存器中，然后在sdo上移出这些数据，同时在sdi上移入从控制器传输的数据(t[7:0])。计数器cnt用于跟踪已发送/接收的位数。当sck空闲时，cnt = 0，d的最高有效位(d[7])保持在sdo线上。一个细微之处在于，sdo只能在时钟下降沿变化，因此sdo输出（即移位寄存器的最高有效位q[7]）通过图e9.9左下角的负边沿触发的qdelayed寄存器延迟了半个时钟周期。

HDL示例 e9.2 用于SPI外设的HDL

<pre>module spi_peripheral(input logic sck, // From controller input logic sdi, // From controller output logic sdo, // To controller input logic reset, // System reset input logic [7:0] d, // Data to send output logic [7:0] q); // Data received logic [2:0] cnt; logic qdelayed; // 3-bit counter tracks when full byte is transmitted always_ff @(negedge sck, posedge reset) if (reset) cnt = 0; else cnt = cnt + 3'b1;</pre>	<pre>// Loadable shift register // Loads d at the start, shifts sdi into bottom on each step always_ff @(posedge sck) q <= (cnt == 0) ? {d[6:0], sdi} : {q[6:0], sdi}; // Align sdo to falling edge of sck // Load d at the start always_ff @(negedge sck) qdelayed = q[7]; assign sdo = (cnt == 0) ? d[7] : qdelayed; endmodule</pre>
---	---



图e9.9 FPGA上SPI外设的模块与时序图

通用异步收发器（UART）
UART（发音为“you-art”）是一种串行I/O外设，可在两个系统之间进行通信而无需发送时钟信号。相反，系统必须事先就使用的数据速率达成一致，并各自在本地生成自己的时钟。因此，由于时钟不同步，这种传输是异步的。尽管这些系统时钟可能存在微小的频率误差和未知的相位关系，但UART仍能实现可靠的异步通信。UART用于RS-232和RS-485等协议中。例如，旧式计算机串行端口采用1969年由电子工业协会推出的RS-232C标准。该标准最初设想连接*data terminal equipment*（DTE），如大型机

计算机到*data communication equipment* (DCE) 如调制解调器。尽管UART与SPI相比速度较慢且容易出现配置错误问题，但这些标准已经存在了很长时间，因此至今仍然很重要。

图e9.10(a)展示了一个异步串行链路。DTE通过TX线向DCE发送数据，并通过RX线接收返回的数据。图e9.10(b)显示了其中一条线路以9600波特的速率发送一个字符。线路在空闲时保持逻辑“1”。每个字符以起始位(0)、7或8个数据位、一个可选的奇偶校验位以及一个或多个停止位(1)的形式发送。最常见的情况是发送起始位、停止位和8个数据位。UART通过检测从空闲到起始的下降沿，在适当的时间锁定传输。尽管7个数据位足以发送ASCII字符，但通常使用8个数据位，因为它们可以传输任意字节的数据。

可选的奇偶校验位允许系统检测传输过程中是否发生比特错误。可配置为*even*或*odd*；*even parity*表示奇偶校验位的选择使得数据和校验位的总集合中1的个数为偶数。换句话说，校验位是数据位的异或结果。接收方随后可以检查是否接收到偶数个1，若否则发出错误信号。*Odd parity*则相反。

常见的选择是1个起始位、8个数据位、无校验位和1个停止位，总共用10个符号来传输一个8位信息字符。因此，信号速率以波特为单位，而非比特/秒。例如，9600波特表示9600符号/秒，即960字符/秒。发送端和接收端必须配置为合适的波特率以及数据位、校验位和停止位的数量，否则数据将会错乱。这对用户来说颇为麻烦，尤其是非技术用户，这也是USB在个人计算机系统中取代UART的原因之一。

典型的波特率包括300、1200、2400、9600、14400、19200、38400、57600和115200。较低的速率在20世纪70年代和80年代用于通过电话线以一系列音调发送数据的调制解调器。在当代系统中，9600和115200是最常用的两种。

波特率给出*signaling*，以符号/秒为单位，而比特率给出*data rate*，以比特/秒为单位。在像SPI这样的简单系统中，每个符号就是一个数据位，因此波特率等于比特率。UART和其他一些信号传输协议除了数据位之外还需要额外的开销位。例如，一个UART每8位数据添加起始位和停止位（即每8位数据对应10个符号），并以9600的波特率运行时，其比特率为 $(9600 \text{ 符号/秒}) \times (8 \text{ 比特/10 符号}) = 7680 \text{ 比特/秒} = 960 \text{ 字符/秒}$ 。

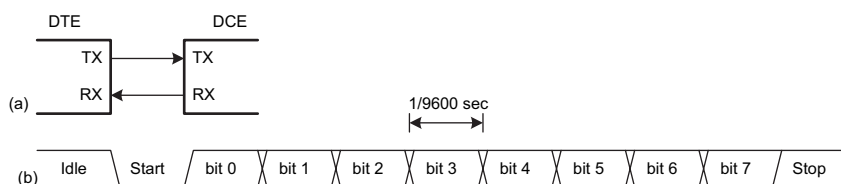


图 e9.10 异步串行链路

在20世纪50年代至70年代，早期自称*phone phreaks*的黑客学会了通过吹出相应音调来控制电话公司的交换机。来自Cap'n Crunch麦片盒中的玩具哨子（图e9.11）发出的2600赫兹音调，可被利用来拨打免费的长途和国际电话。



图 e9.11 Cap'n Crunch Bosun Whistle (摄影: Evrim Sen, 经许可转载。)

握手是指两个系统之间的协商过程。通常，一个系统发出信号表示已准备好发送或接收数据，另一个系统则对该请求进行确认。

常见波特率；在速度无关紧要的场合使用9600，而115200是最快的标准速率，尽管与其他现代串行I/O标准相比仍然较慢。

RS-232标准定义了若干附加信号。请求发送（RTS）和清除发送（CTS）信号可用于*hardware handshaking*。它们可在两种模式之一中运行：流控制或单工。在*flow control*模式下，当DTE准备从DCE接收数据时，它将RTS清零为0。同样，当DCE准备从DTE接收数据时，它将CTS清零为0。某些数据手册使用上划线表示这些信号为低电平有效。在较旧的*simplex*模式下，当DTE准备发送时，它将RTS清零为0。DCE在准备接收传输时，通过将CTS清零作为回应。

某些系统，尤其是通过电话线连接的设备，还会使用数据终端就绪（DTR）、数据载波检测（DCD）、数据集就绪（DSR）和振铃指示（RI）信号来指示设备何时连接到线路。这些信号仍会出现在某些连接器中。

最初的RS-232标准推荐使用庞大的25针DB-25连接器，但PC将其精简为公头9针DE-9连接器，其引脚排列如图e9.12(a)所示。电缆线通常直连，如图e9.12(b)所示。然而，当直接连接两个DTE时，可能需要使用图e9.12(c)所示的*null modem*电缆来交换RX和TX并完成握手。更令人困扰的是，有些连接器是公头，有些是母头。总之，通过RS-232连接两个系统可能需要一大箱电缆和一定程度的猜测，这也解释了为何转向USB。幸运的是，嵌入式系统通常使用简化的3线或5线设置，包括GND、TX、RX，以及可能的RTS和CTS。

RS-232在电气上以3至15 V表示0，以-3至-15 V表示1；这称为*bipolar*信号传输。收发器将UART的数字逻辑电平转换为RS-232所期望的正负电平，并提供静电放电保护，以防止用户在插入电缆时损坏串口。MAX3232E是一款流行的收发器，兼容3.3 V和5 V数字逻辑。它包含一个电荷泵，结合外部电容器，可从单个低压电源产生 ± 5 V输出。一些用于嵌入式系统的串行外设省略了收发器，仅使用0 V表示0，使用3.3或5 V表示1；请查阅数据手册！

FE310有两个UART，分别命名为UART0和UART1。UART0可配置在引脚16和17上工作；UART1在引脚18和23上工作。要将这些引脚用作UART而非GPIO，其对应的iof_sel位应设置为0（以选择IOF0），并且

将*iof_en*位设置为1以启用外设控制。与SPI类似，FE310必须首先配置端口。与SPI不同的是，读写操作可以独立进行，因为任一系统都可以在不接收的情况下发送，反之亦然。UART0的寄存器如表e9.6所示。

配置UART时，首先设置波特率。UART使用板载TileLink总线时钟*tlclk*作为其时钟源。对于FE310-G002，该总线时钟默认配置为与处理器时钟*coreclk*相同，频率为16 MHz。此时钟信号必须进行分频以产生所需的波特率。最终波特率由公式9.1给出：

$$f_{baud} = \frac{f_{in}}{div + 1}$$

(e9.1)

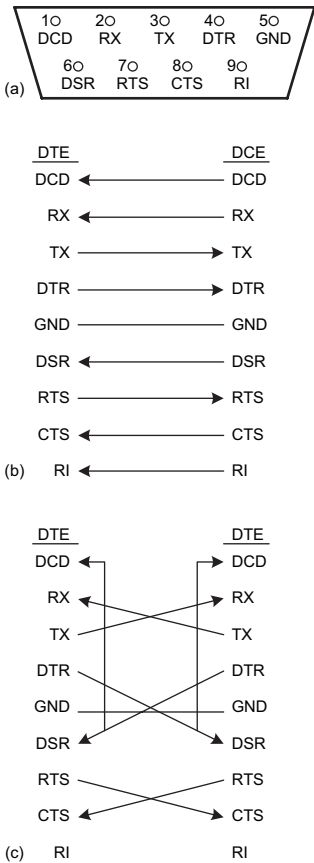
FE310 UART外设仅支持8-N-1和8-N-2协议配置。两种协议均支持8个数据位且无校验位，数据包可配置为具有一个停止位（8-N-1）或两个停止位（8-N-2）。停止位配置通过*txctrl*寄存器中的*nstop*字段设置。默认情况下，*nstop* = 0，即设置外设使用一个停止位。

数据通过*txdata*和*rxdata*进行发送和接收。寄存器，分别。发送和接收寄存器均由8条目FIFO缓冲器缓冲。要发送数据，请检查*txdata*寄存器的*full*位是否为0，这表示FIFO缓冲器中有空间可写入新数据。然后，向*txdata*的数据字段写入一个字节。要读取数据，请读取*rxdata*寄存器并检查*empty*位是否为0，以确认数据字段中的字节有效。

示例 e9.4 与PC的串行通信

为FE310开发一个电路和C程序，使其通过串口以115200波特率、8个数据位、1个停止位、无校验的方式与PC通信。PC应运行PuTTY¹等控制台程序来读写串口。该程序应要求用户输入一个字符串，然后显示用户输入的内容。

解决方案 图e9.13(a)展示了串行链路的基本原理图，说明了电平转换和布线的问题。由于很少有PC仍配备物理串行端口，我们使用来自plugable.com的Plugable USB转RS-232 DB9串行适配器（如图e9.14所示）来提供与PC的串行连接。该适配器连接到一个母头DE-9连接器，该连接器焊接在导线上，导线连接至收发器，收发器



图e9.12 DE-9公头线缆 (a) 引脚排列, (b) 标准接线, 和 (c) 零调制解调器接线

表 e9.6 UART 内存映射寄存器

	...
0x10013018	div
	...
0x1001300C	rxctrl
0x10013008	txctrl
0x10013004	rxdata
0x10013000	txdata
	...

经SiFive FE310- © 2019 SiFive, Inc. 许可改编并印刷，源自表55。

G002 Manual

¹ PuTTY is available for free download at www.putty.org.

将电压从双极性RS-232电平转换为FE310的3.3V电平。FE310和PC均为DTE设备，因此电路中的TX和RX引脚必须交叉连接。FE310的RTS/CTS握手信号未被使用，DE9连接器上的RTS和CTS短接在一起，以便PC自行握手。

图e9.13(b)展示了一种更简便的方法，使用Adafruit 954 USB转TTL串口线。该线缆直接兼容3.3 V电平。
要配置PuTTY以使用串行链路，请将*Connection type*设置为Serial，并将*Speed*设置为115200。将*Serial line*设置为操作系统分配给串口转USB适配器的COM端口。在

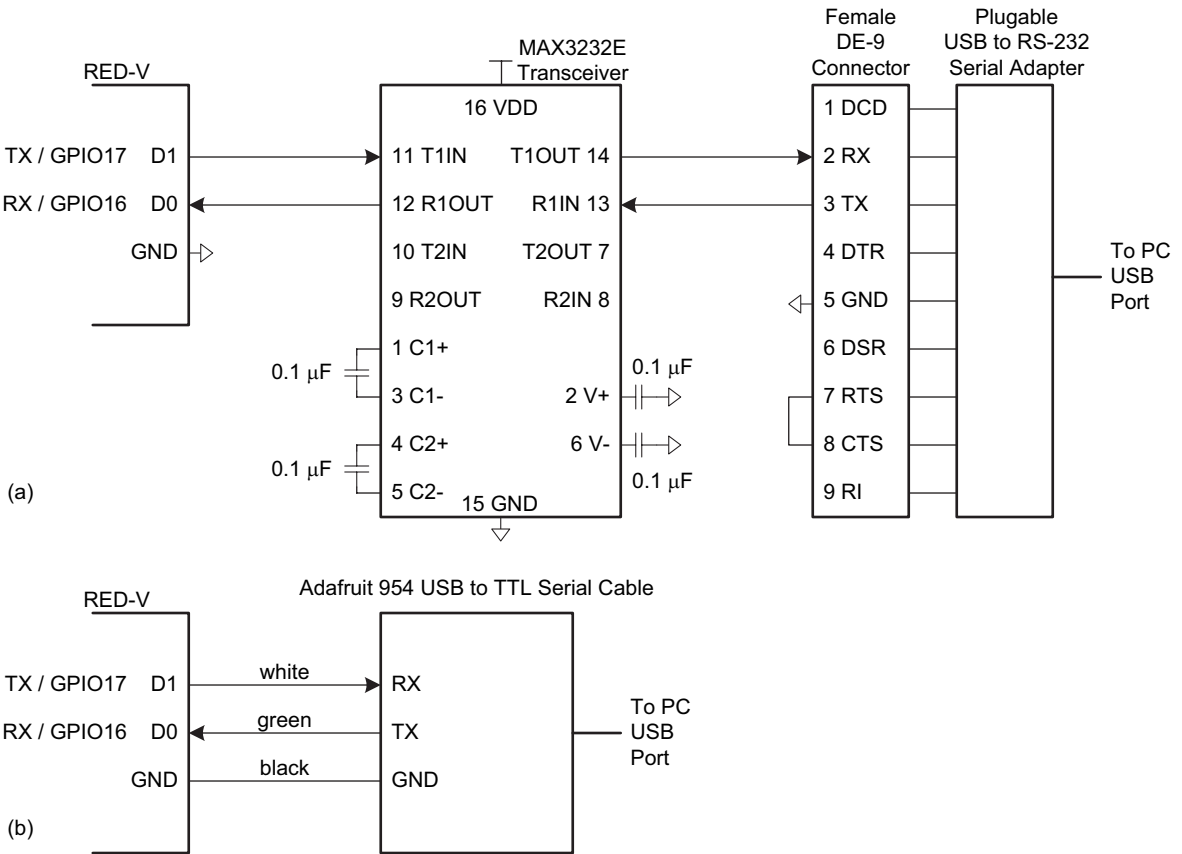


图 e9.13 串行通信链路示意图: (a) 通过RS-232的串行通信, (b) 使用USB转TTL串行线的串行通信

在Windows系统中，可以在设备管理器中找到该信息；例如，它可能是COM3。在`Connection` → `Serial`选项卡下，将流控制设置为NONE或RTS/CTS。在终端选项卡下，将本地回显设置为强制开启，以便在终端中键入字符时显示这些字符。

EasyREDVIO.h中的串口设备驱动代码如代码示例e9.5所示。终端程序中的回车键对应一个回车符，在C语言中表示为`\r`，其ASCII码为0x0D。要在打印时换行到下一行开头，需同时发送`\n`和`\r`（换行和回车）字符。² `uartInit`函数按上述方式配置UART。`getCharSerial`和`putCharSerial`分别通过UART从终端读取字符和向终端写入字符（代码示例e9.5）。



图 e9.14 可插拔 USB 转 RS-232 DB9 串口适配器

Code Example e9.5 READING AND WRITING CHARACTERS (CHARS) TO A TERMINAL USING A UART

```
void uartInit(uint32_t baud) {
    uint32_t div = 16000000/baud-1;    // 16 MHz tileclock
    pinMode(16, GPIO_IOF0);
    pinMode(17, GPIO_IOF0);

    UART0->div.div = div;               // Set clock divisor
    UART0->txctrl.txen = 1;             // Enable transmitter
    UART0->txctrl.nstop = 1;            // Set one stop bit
    UART0->rxctrl.rxen = 1;             // Enable receiver
}

uint8_t getCharSerial(void) {
    uart_rxdata_bits rxdata;           // Create temporary variable to store register

    while(1) {
        rxdata = UART0->rxdata;        // Read register exactly once
        if(!rxdata.empty) {
            return (uint8_t)rxdata.data; // Check to see if the data is valid
        }
    }
}

void putCharSerial(uint8_t c) {
    while(UART0->txdata.full);          // Wait until ready to transmit
    UART0->txdata.data = c;
}
```

代码示例 e9.6 中的主函数演示了如何使用 `putStrSerial` 和 `getStrSerial` 函数向控制台打印内容以及从控制台读取内容。

² PuTTY prints correctly even if the `\r` is omitted.

Code Example e9.6 READING AND WRITING STRINGS TO A TERMINAL USING A UART

```
#include "EasyREDVIO.h"

#define MAX_STR_LEN 80

void getStrSerial(char *str) {
    int i = 0;
    do {
        // Read an entire string until detecting
        str[i] = getCharSerial(); // Carriage return
    } while ((str[i++] != '\r') && (i < MAX_STR_LEN)); // Look for carriage return
    str[i-1] = 0; // Null-terminate the string
}

void putStrSerial(char *str) {
    int i = 0;
    while (str[i] != 0) { // Iterate over string
        putcharSerial(str[i++]); // Send each character
    }
}

int main(void) {
    char str[MAX_STR_LEN];

    uartInit(115200); // initialize UART with baud rate

    while(1) {
        putStrSerial("Please type something: \r\n");
        getStrSerial(str);
        putStrSerial("You typed: ");
        putStrSerial(str);
        putStrSerial("\r\n");
    }
}
```

在PC上通过C程序与串口通信有些麻烦，因为串口驱动库在不同操作系统间并未标准化。而其他编程环境如Python、MATLAB或LabVIEW则让串口通信变得轻松简单。

9.3.6 定时器

嵌入式系统通常需要测量时间。例如，微波炉需要一个计时器来跟踪一天中的时间，另一个计时器来测量烹饪时长。它可能还需要一个计时器来生成脉冲以驱动转盘电机，以及第四个计时器通过仅在每秒的一部分时间内激活微波能量来控制功率设置。

FE310 具有一个系统定时器，带有一个 64 位自由运行计数器，该计数器根据外部提供的时钟信号递增。在 RED-V 上，该时钟源是一个 32.768 kHz 振荡器（方便地表示为 2^{15} Hz）。图 e9.16 显示了系统定时器的内存映射。它位于核心本地中断器（CLINT）模块内。mtime 包含计数器的 64 位当前值。该值可读可写；因此，要重新启动

定时器中，可以写入一个零。mtimecmp是一个64位寄存器，包含定时器比较值，而msip是机器模式软件中断寄存器。当计数器达到mtimecmp中的值时，msip寄存器的最低有效位被设置为1。使用msip和mtimecmp寄存器是检查延迟是否已发生的有效方法。表e9.7显示了这些寄存器的内存地址。

如果需要额外的定时器，PWM模块（参见第9.3.7.2节）提供了额外的计数器，可用于测量精确的延迟。

示例 e9.5 闪烁的LED

编写一个程序，使RED-V上的状态LED以每秒5次的频率闪烁，持续4秒。

解决方案 EasyREDVIO中的延迟函数（参见代码示例e9.7）使用定时器比较通道创建指定毫秒数的延迟。

代码示例 e9.7 延迟函数

```
#define MTIME_CLK_FREQ 32768 // RTC频率，单位为Hzvolatile uint64_t *mtime = (uint32_t *) 0x0200BFF8;void delay(int ms) { uint64_t doneTime = *mtime + (ms*MTIME_CLK_FREQ)/1000; while (*mtime < doneTime); // 等待直到时间到达}
```

GPIO5 (D13) 驱动 RED-V 板上的活动 LED。代码示例 e9.8 中的程序将该引脚设置为输出。然后通过一系列数字写入操作，以 200 毫秒的重复频率（5 Hz）关闭和打开 LED。

代码示例 e9.8 闪烁活动LED

```
#include "EasyREDVIO.h"void main(void) { uint32_t i; pinMode(D13, OUTPUT); // status led as output for(i = 0; i < 20; i++) { delay(100); digitalWrite(D13, 0); // turn led off delay(100); digitalWrite(D13, 1); // turn led on } }
```

9.3.7 模拟I/O

现实世界是一个模拟场所。许多嵌入式系统需要模拟输入和输出来与外界交互。它们使用

表 e9.7 系统定时器寄存器

	...
0x0200BFFC	mtime (hi)
0x0200BFF8	mtime (lo)
	...
0x02004004	mtimecmp (hi)
0x02004000	mtimecmp (lo)
	...
0x02000000	msip
	...

经SiFive FE310-G002 Manual, © 2019 SiFive, Inc.许可，改编并印刷自表24。

模数转换器（ADC）用于将模拟信号量化为数字值，而数模转换器（DAC）则执行相反操作。图e9.15展示了这些组件的符号。此类转换器的特性由其分辨率、动态范围、采样率和精度决定。例如，一个ADC可能具有 $N = 12$ 位分辨率，范围 V_{ref}^- 至 V_{ref}^+ 为0至5V，采样率为 $f_s = 44\text{ kHz}$ ，精度为 ± 3 个最低有效位（lsb）。采样率也以每秒采样数（sps）表示，其中 $1\text{ sps} = 1\text{ Hz}$ 。模拟输入电压 $V_{in}(t)$ 与数字采样值 $X[n = t / f_s]$ 之间的关系为

$$X[n] = 2^N \frac{V_{in}(t) - V_{ref}^-}{V_{ref}^+ - V_{ref}^-} \tag{e9.2}$$

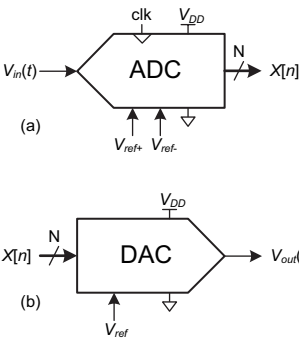


图 e9.15 ADC 和 DAC 符号

例如，2.5 V（满量程的一半）的输入电压将对应 $2^{12}/2$ （最大值的一半），即输出为 $100000000000_2 = 0x800 = 2^{11} = 2048$ ，不确定度高达3个LSB。

类似地，一个DAC可能具有 $N = 16$ 位分辨率，满量程输出范围为 $V_{ref} = 2.56\text{ V}$ 。它产生的输出为

$$V_{out}(t) = \frac{X[n]}{2^N} V_{ref} \tag{e9.3}$$

许多微控制器内置了性能适中的ADC。对于更高性能的需求（例如16位分辨率或超过1 MHz的采样率），通常需要使用连接到微控制器的独立ADC。内置DAC的微控制器较少，因此必须使用独立的芯片将数字值转换为模拟电压。然而，微控制器常通过一种称为脉宽调制（PWM）的技术产生模拟输出。

数模转换

FE310没有通用DAC。本节介绍使用外部DAC进行数模转换，并说明如何通过并行和串行端口将FE310与其他芯片连接。下一节将使用PWM实现相同的结果。

某些DAC通过并行接口以 N 条线接收 N 位数字输入，而另一些则通过串行接口（如SPI）接收。部分DAC需要正负电源电压，其他则仅需单电源供电。有些支持灵活的电源电压范围，有些则要求特定电压。输入逻辑电平应与数字源兼容。某些DAC

产生与数字输入成比例的电压输出，而其他则产生电流输出；可能需要一个运算放大器来将此电流转换为所需范围内的电压。

在本节中，我们使用Linear Technology LTC1450 12位并行DAC和Microchip MCP4801 8位串行DAC。两者均产生电压输出，由单个5至15V电源供电，使用 $V_{IH} = 2.4V$ ，因此与3.3V I/O兼容，采用DIP封装，便于在面包板上使用，且易于操作。LTC1450根据增益设置，输出范围为0至2.048V或0至4.095V，功耗为2mW，采用24引脚封装，建立时间为4 μ s，支持250k样本/秒的输出速率。其数据手册可在analog.com获取。MCP4801输出范围为0至2.048V或0至4.096V，功耗低于2mW，采用8引脚封装，建立时间为4.5 μ s。其SPI最高工作频率为20MHz。数据手册可在microchip.com获取。

示例 e9.6 使用外部DAC的模拟输出

设计一个电路并编写软件，使用RED-V、LTC1450和MCP4801生成正弦波和三角波的简易信号发生器。

解决方案 电路如图e9.16所示。本例中使用了两片DAC芯片。两个DAC均采用5V电源供电，并配有0.1 μ F去耦电容以降低电源噪声。

LTC1450 DAC有12个数据输入引脚D0至D11，用于指定在VOUT上生成的模拟电压。在本示例中，我们仅使用8位精度，因此将最低四位D0至D3接地。要将数据加载到DAC中，RED-V将所需值置于D4至D11上。随后，RED-V将低电平有效的写入（WR）引脚拉低，将数据写入DAC。CLR接VCC，因为我们无需清除输入数据锁存器。低电平有效的加载DAC信号LDAC接GND，以便每次WR变为低电平时加载数据。

MCP4801通过SPI1连接到RED-V。除了标准的SPI信号外，MCP4801还有一个模拟输出电压引脚（VOUT）和两个低电平有效的控制输入：硬件关断（SHDN）和锁存DAC（LDAC）。当不需要输出值时，SHDN用于关闭输出驱动电路以节省功耗。LDAC在低电平时锁存输入值。要向MCP4801发送数据，需通过SPI发送一个16位值：位11到位4保存D7到D0；位13是增益选择（设为1时为1倍，设为0时为2倍）；位12控制SHDN（0关闭输出，1允许VOUT驱动模拟值）。在此情况下，SHDN由软件控制，因此在电路中保持悬空（不驱动）。

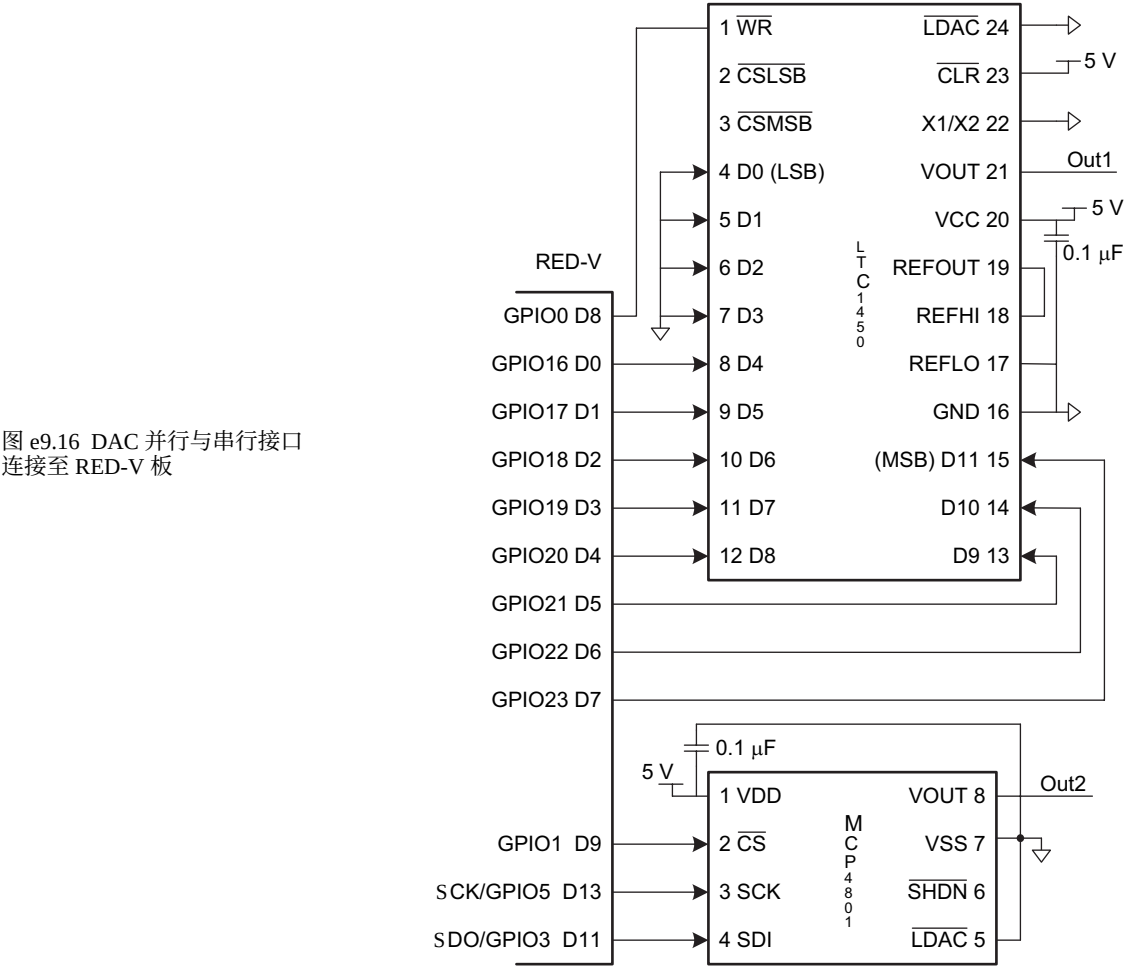


图 e9.16 DAC 并行与串行接口
连接至 RED-V 板

驱动两个DAC的程序如代码示例e9.9所示。该程序将8个并行端口引脚配置为输出，同时将GPIO0配置为输出以驱动LTC1450上的WR信号，并将GPIO1配置为输出以驱动MCP4801上的片选信号。它将SPI初始化为500 kHz。initWaveTables预计算正弦波和三角波的采样值数组。随后更新串行DAC。然后，程序进行延迟，直到定时器指示下一个采样点的时间。生成波形的最大频率由genWaves函数中发送每个点的时间决定，该时间受限于SPI传输时间。

代码示例 e9.9 使用 DAC 生成正弦波

```

#include "EasyREDVIO.h"
#include <math.h> // required to use the sine function

#define NUMPTS 64
int sine[NUMPTS], triangle[NUMPTS];

#define SHDNn_Pos 12
#define Gain_Pos 13

int parallelPins[8] = {D0, D1, D2, D3, D4, D5, D6, D7};

void initWaveTables(void) {
    int i;
    for (i = 0; i < NUMPTS; i++) {
        sine[i] = 127*(sin(2*3.14159*i/NUMPTS) + 1); // 8-bit scale
        if (i < NUMPTS/2) triangle[i] = i*255/NUMPTS; // 8-bit scale
        else triangle[i] = 254 - i*255/NUMPTS;
    }
}

void genWaves(int freq) {
    int i, j;
    int delay_cycles = MTIME_CLK_FREQ/(NUMPTS*freq);

    for (i = 0; i < 2000; i++){
        for (j = 0; j < NUMPTS; j++) {
            uint64_t doneTime = *mtime + delay_cycles; // Set sample period

            // Load serial DAC
            digitalWrite(1, 0); // enable chip (chip select: CS = 0)
            // Set SHDNn to active (bit 12) and gain to 1 (bit 13)
            volatile uint16_t sine_samp_dac = ((uint16_t) sine[j] << 4) \
                | (1 << SHDNn_Pos) | (1 << Gain_Pos);
            spiSendReceive16(sine_samp_dac);
            digitalWrite(1, 1); // disable chip (chip select: CS = 1)

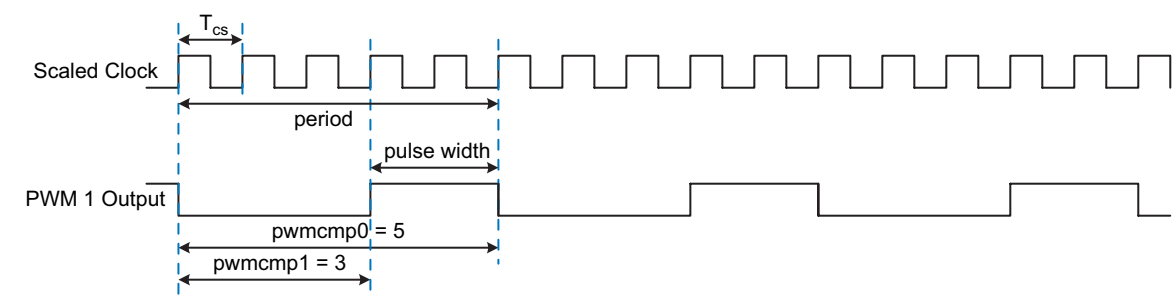
            // Load parallel DAC
            digitalWrite(0, 1); // No load while changing inputs
            digitalWritees(parallelPins, 8, triangle[j]);
            digitalWrite(0, 0); // Load new points into DACs
            while(*mtime < doneTime); // Wait for mtime_cmp to hit
        }
    }
}

int main(void) {
    pinsMode(parallelPins, 8, OUTPUT); // Set pins connected to the AD558 as outputs
    pinMode(0, OUTPUT); // Make pin 0 an output to control LOAD
    pinMode(1, OUTPUT); // Make pin 1 an output to control CE
    spiInit(15, 0, 0); // Initialize the SPI
    initWaveTables();
    genWaves(100);
}

```

脉宽调制

数字系统产生模拟输出的另一种方式是使用 *pulse-width modulation* (PWM)，其中周期性输出在部分周期内为高电平，其余时间为低电平。占空比是指脉冲为高电平的时间占整个周期的比例（脉冲宽度/周期），如图e9.17所示。输出的平均值为



图e9.17 脉宽调制（PWM）信号

与占空比成正比。例如，如果输出在0到3.3 V之间摆动，且占空比为25%，则平均值将为 $0.25 \times 3.3 = 0.825\text{ V}$ 。对PWM信号进行低通滤波可消除振荡，并保留具有所需平均值的信号。因此，当脉冲频率远高于感兴趣的模拟输出频率时，PWM是产生模拟输出的有效方法。PWM的其他应用包括生成方波音频音调，以及以部分功率或亮度对电机或灯光进行数字控制。

FE310 具有三个 PWM 外设，如表 e9.3 所示，每个 PWM 有四个 PWM 输出，总共提供 12 个 PWM 输出。PWM0 的输出具有 8 位精度，而 PWM1 和 PWM2 具有 16 位精度。在本节中，我们演示如何使用 PWM2，但配置和使用其他两个 PWM 外设的步骤类似。PWM2 有四个输出（PWM2_PWM0、PWM2_PWM1、PWM2_PWM2、PWM2_PWM3），这些输出可通过引脚 GPIO10-13 使用引脚功能 IOF1 获得。

表 e9.8 PWM2 配置寄存器

0x1002502C	pwmcmp3
0x10025028	pwmcmp2
0x10025024	pwmcmp1
0x10025020	pwmcmp0
...	...
0x10025010	pwms
0x1002500C	...
0x10025008	pwmcount
0x10025004	...
0x10025000	pwmcfg
	...

经SiFive FE310-G002 Manual表89
改编并获许可印刷，© 2019 SiFive,
Inc.

PWM具有多种波形生成模式，但我们重点关注生成如图e9.17所示的PWM波形。为此，需将外设配置为重复模式，其中比较器0（pwmcmp0）设置周期，比较器1（pwmcmp1）设置低电平时间。这些时间以经过分频的时钟周期 T_{cs} 为单位。例如，如图e9.17所示，若分频时钟周期为 $0.5\mu\text{s}$ （2 MHz），且 $pwmcmp0 = 5$ ，则PWM2_PWM1（引脚11）将以 $5 \times 0.5\mu\text{s} = 2.5\mu\text{s}$ （400 kHz）的周期振荡。若 $pwmcmp1 = 3$ ，则占空比为 $1 - (3/5) = 40\%$ 。

表e9.8显示了PWM2寄存器的内存映射。在本节中，我们描述了配置PWM1_PWM1输出所需的步骤；其他PWM及其输出采用类似的过程进行配置。

表 e9.9 显示了 PWM 配置寄存器 pwmcfg 中的位域。请注意，大多数位在系统复位时不会被清除。

表 e9.9 PWM 配置寄存器字段

PWM Configuration Register (pwmcfg)				
Register Offset		0x0		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	pwmscale	RW	X	PWM Counter scale
[7:4]	Reserved			
8	pwmsticky	RW	X	PWM Sticky - disallow clearing pwmcmp _X ip bits
9	pwmzerocmp	RW	X	PWM Zero - counter resets to zero after match
10	pwmdeglitch	RW	X	PWM Deglitch - latch pwmcmp _X ip within same cycle
11	Reserved			
12	pwmalways	RW	0x0	PWM enable always - run continuously
13	pwmone-shot	RW	0x0	PWM enable one shot - run one cycle
[15:14]	Reserved			
16	pwmcmp0center	RW	X	PWM0 Compare Center
17	pwmcmp1center	RW	X	PWM1 Compare Center
18	pwmcmp2center	RW	X	PWM2 Compare Center
19	pwmcmp3center	RW	X	PWM3 Compare Center
[23:20]	Reserved			
24	pwmcmp0gang	RW	X	PWM0/PWM1 Compare Gang
25	pwmcmp1gang	RW	X	PWM1/PWM2 Compare Gang
26	pwmcmp2gang	RW	X	PWM2/PWM3 Compare Gang
27	pwmcmp3gang	RW	X	PWM3/PWM0 Compare Gang
28	pwmcmp0ip	RW	X	PWM0 Interrupt Pending
29	pwmcmp1ip	RW	X	PWM1 Interrupt Pending
30	pwmcmp2ip	RW	X	PWM2 Interrupt Pending
31	pwmcmp3ip	RW	X	PWM3 Interrupt Pending

经许可转载自SiFive FE310-G0002 Manual, 表91, © 2019 SiFive, Inc.

因此，明智的做法是先将所有位重置为0，然后始终向pwm_{en}写入1，并向pwm_{zerocmp}写入1，以配置PWM生成重复波形，其周期由pwm_{cmp0}设定。

缩放后的时钟频率 f_{scaled} 是基础总线时钟频率 $f_{\text{base}} = 16 \text{ MHz}$ 除以 $2^{\text{pwm}\text{scale}}$ 的结果，其中 pwmscale 是 pwmcfg 寄存器中一个 4 位数值，范围为 0 到 15。PWM 频率为 $f_{\text{pwm}} = f_{\text{scaled}} / \text{pwm}\text{cmp0} = f_{\text{base}} / (\text{pwm}\text{cmp0} \times 2^{\text{pwm}\text{scale}})$ 。如上所述，占空比为 $1 - (\text{pwm}\text{cmp1} / \text{pwm}\text{cmp0})$ 。对于 pwmscale 和 $\text{pwm}\text{cmp0}$ ，存在多种可能的选择以实现所需的 PWM 频率。然而，在 $\text{pwm}\text{cmp0}$ 为无符号 16 位数值（即不超过 65,535）的约束下，当 pwmscale 尽可能小且 $\text{pwm}\text{cmp0}$ 尽可能大时，PWM 频率分辨率（期望频率与实际频率之间的误差）最佳。

示例 e9.7 脉宽调制 (PWM)

选择 `pwm scale` 和 `pwm cmp0` 以 1.2 Hz 频率闪烁 LED。重复该问题以生成 1190 Hz 的音调。

解决方案 为了说明这一点，假设我们希望以 $f_{\text{pwm}} = 1.2 \text{ Hz}$ 的频率闪烁一个 LED。 $f_{\text{pwm}} = f_{\text{base}} / (\text{pwm cmp0} \times 2^{\text{pwm scale}})$ 。因此，选择 `pwm scale = 8` 和 `pwm cmp0 = 52083.33`，以在 $f_{\text{scaled}} = 16 \text{ MHz} / 2^8 = 62.5 \text{ KHz}$ 下获得所需频率。`pwm cmp0` 是一个 16 位寄存器，因此我们必须四舍五入到 52083，得到实际 $f_{\text{pwm}} = 16 \text{ MHz} / (52083 \times 2^8) = 1.20000768 \text{ Hz}$ ，这非常接近所需频率，并且与典型石英晶体时钟参考的百万分之十精度相当。

另一方面，假设我们需要一个 1190 Hz 的输出。如果不改变 `pwm scale`，`pwm cmp0` 需要设为 52.521。四舍五入到 53 后，实际频率 $f_{\text{pwm}} = 16 \text{ MHz} / (53 \times 2^8) = 1179.2 \text{ Hz}$ ，误差约为 10 Hz，即 0.91%。如果需要更精确的输出频率，可以将 `pwm scale` 减小到 0，并将 `pwm cmp0` 增加到 13445，得到 $f_{\text{pwm}} = 16 \text{ MHz} / (13445 \times 2^0) = 1190.03 \text{ Hz}$ 。

一个 PWM 设备驱动程序可能包含 ``pwmInit()` 和 ``pwm(int freq, float duty)` 函数。``pwmInit` 会为 PWM 外设设置合适的引脚，并在 ``pwm cfg` 寄存器中配置相应的位。``pwm` 函数则会选择合适的 ``pwm scale``、``pwm cmp0`` 和 ``pwm cmp1``，以生成具有指定频率和占空比的波形。编写这些函数与编写 SPI 或 UART 设备驱动程序类似；具体细节留给读者作为练习。

模数转换

许多微控制器至少有一个内置 ADC，但 FE310 没有。本节介绍使用外部转换器进行模数转换，类似于前一节所述的外部 DAC。

示例 e9.8 使用外部 ADC 的模拟输入

使用 SPI 将 10 位 MCP3002 A/D 转换器连接到 FE310，并打印输入值。设置满量程电压为 3.3 V。在网上搜索数据手册以获取完整的操作详情。

解决方案 图 e9.18 展示了 FE310 与 MCP3002 ADC 之间的连接示意图，代码示例 e9.10 给出了驱动代码。MCP3002 使用 VDD 作为其满量程参考：即 VDD（引脚 8）连接到 3.3 V。它可接受 3.3 至 5.5 V 的供电电压；我们选择 3.3 V。该 ADC

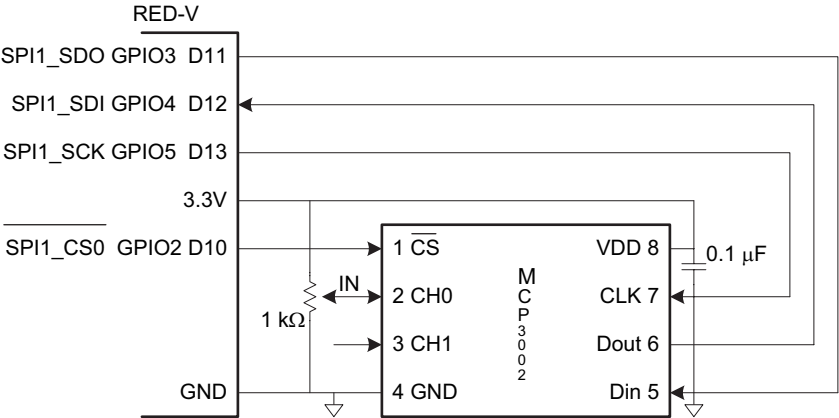


图 e9.18 使用外部ADC读取模拟输入

有两个输入通道，CH0和CH1。我们将通道0连接到一个电位器（图中未显示），通过旋转该电位器来调节输入电压在0到3.3V之间。

FE310代码（参见代码示例e9.10）初始化SPI并反复读取和打印采样值。根据数据手册，FE310必须通过SPI发送16位数值0x6000以读取CH0，并将在16位结果的低10位中接收到10位结果。由于无法直接设置FE310传输16位帧，我们可以组合两个8位数据包，且期间不拉高片选线。尽管SPI外设具有自动控制片选线的选项，但此处我们手动将GPIO2配置为输出，并在传输开始时（将片选线写为0）和传输结束时（将片选线写为1）适当地切换该引脚。

代码示例 e9.10 与ADC接口的代码

```
#include "EasyREDVIO.h"

int main(void) {
    uint8_t sample;
    spiInit(15, 0, 0); // Initialize the SPI
                        // Clock divisor of div = 15, CPOL = 0, CPHA = 0
    pinMode(D10, OUTPUT);
    while(1) {
        digitalWrite(D10, 0);
        spiSendReceive('0x60');
        sample = spiSendReceive('0x00');
        digitalWrite(D10, 1);
        printf("Read %d\n", sample);
        delay(200);
    }
}
```

9.3.8 中断

到目前为止，我们一直依赖*polling*，即程序持续检查某个值，直到事件发生，例如UART上有数据到达或定时器达到其比较值。这可能会浪费处理器的功耗，并且使得编写在等待事件发生的同时执行有意义工作的程序变得困难。

大多数微控制器支持*interrupts*。当事件发生时，微控制器停止正在执行的程序，并跳转到响应中断的中断处理程序。处理完中断后，处理器返回用户程序，并在被中断处无缝继续执行。中断是第6.6.2节讨论的硬件异常。

FE310 具有一个核心本地中断控制器（CLINT），用于处理定时器和软件中断。软件中断用于处理器间通信和调试。FE310 还拥有一个平台级中断控制器（PLIC），用于收集来自其他外设的中断。在多处理器系统中，PLIC 将外设中断路由到合适的处理器进行处理。

示例e9.9展示了如何使用中断而非轮询来使LED闪烁。

示例 e9.9 使用定时器中断闪烁LED

我们使用CLINT在FE310上配置本地中断。对于RED-V RedBoard和RED-V Thing Plus上使用的FE310-G002芯片，关于如何使用中断的信息见 *FE310-G002 Manual* 的第8至10章。通过CLINT配置本地中断的基本流程如下。

1. 编写一个陷阱处理程序，用于在触发中断或异常时处理执行。该陷阱处理程序的主要目的是确定触发的中断或异常类型，然后执行相应的操作。
2. 配置 *mtvec*（一个控制和状态寄存器），设置陷阱处理程序的地址以及模式（直接模式或向量模式）。
3. 启用特定中断（例如来自定时器的中断）。
4. 全局启用所有中断。

在定义常量和函数指针数组后，代码声明了全局陷阱处理函数 `handle_trap()`，如代码示例e9.11所示。每当触发陷阱（中断或异常）时，都会调用该函数。其任务是确定哪个陷阱触发了调用，并跳转到正确的中断或异常处理程序。陷阱处理程序执行两项任务。首先，它使用掩码（`MCAUSE_INT_MASK`）检查 `mcause` 寄存器的最高有效位。

该字段指示陷阱是中断（由核外部设备产生）还是异常（由核内部产生）。mcause 的结构如表 e9.10 所示，中断和异常代码的列表如表 6.6 所示。然后，它使用一个额外的掩码（MCAUSE_CODE_MASK）来确定陷阱代码，并根据 interrupt_handler 或 exception_handler 函数指针数组的索引跳转到相应的中断或异常处理程序。

代码示例 e9.11 设置陷阱处理程序

```
// Function pointer arrays for interrupt and exception handlers
#define MAX_INTERRUPTS 16
void (*interrupt_handler[MAX_INTERRUPTS])();
void (*exception_handler[MAX_INTERRUPTS])();

// Masks for isolating interrupt vs. exception and the relevant code
#define MCAUSE_INT_MASK 0x80000000 // If [31] = 1 interrupt, else exception
#define MCAUSE_CODE_MASK 0x7FFFFFFF // low bits show code

// Declaration for interrupt handler. Declared with attribute interrupt which
// maps to GCC helper function.
void handle_trap(void) __attribute__((interrupt));

// Define trap handler
void handle_trap() {
    unsigned long mcause_value = read_csr(mcause);
    if (mcause_value & MCAUSE_INT_MASK) {
        // Branch to interrupt handler here
        // Index into 32-bit array containing addresses of functions
        interrupt_handler[mcause_value & MCAUSE_CODE_MASK]();
    }
    else {
        // Branch to exception handler here
        exception_handler[mcause_value & MCAUSE_CODE_MASK]();
    }
}
```

接下来，我们为定时器定义一个*interrupt service routine*（ISR）。这是一个包含我们希望在每次定时器中断时执行的指令的函数。在此示例中，我们称此函数为timer_handler()。它读取驱动板载LED（D13/GPIO 5）的GPIO引脚的当前值，并使用digitalWrite()切换状态。

表 e9.10 mcause 寄存器字段

Bits	Field Name	Description
[9:0]	Exception Code	A code identifying the most recent exception
[30:10]	Reserved	
31	Interrupt	1 if trap was caused by an interrupt; 0 otherwise

经许可转载自SiFive FE310-G0002 Manual, © 2019 SiFive, Inc.的表22。

然后，它通过调用`reset_timer()`重置定时器，将`mtime`寄存器中的当前计数设置为0，并重置下一次中断应触发的计数值。

代码示例 e9.12 定时器中断服务程序及重置定时器的函数

```
void timer_handler() { volatile int pin_val = (GPIO0->output_val >> D13) & 1; // 读取当前输出状态 if(pin_val) digitalWrite(D13, LOW); else digitalWrite(D13, HIGH); reset_timer(MTIME_CLK_FREQ / (2 * BLINK_FREQ)); } void reset_timer(int count_val) { *MTIME = 0; *MTIMECMP = count_val; }
```

与本章中使用的其他寄存器不同，大多数与CLINT相关的寄存器（如`mtvec`、`mie`和`mstatus`）并非内存映射。这些寄存器被称为 *control and status registers*（CSRs）。要操作CSR，我们必须使用RISC-V汇编指令CSR读取（`csrr`）和CSR写入（`cswr`）。这些指令可以方便地封装在C宏中，以便我们更轻松地与它们交互。

代码示例 e9.13 用于写入和读取CSR的宏

```
// 用于读取和写入控制和状态寄存器（CSR）的宏#define read_csr(reg) ({ unsigned long __tmp; \asm volatile ("csrr %0, " #reg : "=r"(__tmp)); \ __tmp; })#define write_csr(reg, val) ({ \asm volatile ("cswr " #reg ", %0" :: "rK"(val)); })
```

设置陷阱处理程序后，我们将其地址放入 `mtvec` 寄存器来注册它。其结构如表e9.11所示。`mtvec`是一个32位寄存器，其中[31:2]位保存陷阱处理函数地址的[31:2]位（[1:0]位自动假定为0，因为指令必须在内存中字对齐）。`mtvec`的[1:0]位则用于配置异常是以直接模式还是向量模式处理。在直接模式下，无论触发何种中断或异常，我们都跳转到 `mtvec[31:2]`指示的函数地址。这是我们在此处使用的模式。在向量模式下，我们根据触发的中断类型跳转到不同的内存地址。

配置陷阱处理程序并设置定时器中断服务例程后，我们最后通过设置机器中断启用寄存器`mie`中的第7位（机器定时器中断启用位MTIE）来启用机器定时器中断，并通过全局启用中断来

表 e9.11 mtvec 寄存器字段

Bits	Field Name	Description
[1:0]	MODE	Sets the interrupt processing mode to direct (00) or vectored (10)
[31:2]	BASE[31:2]	Base address of the trap_handler

经SiFive FE310-G0002 Manual表18改编并获许可印刷，© 2019 SiFive, Inc.

代码示例 e9.14 通过写入 mtvec 注册陷阱处理函数的函数

```
void register_trap_handler(void *func) {    // 将 mtvec[31:2] 设置为中断处理函数地址    // 最低两位无意义，因为指令按4字节对齐    // 将 mtvec[1:0] 设置为 00 以启用直接模式    write_csr(mtvec, ((unsigned long) func) & ~(0b11));

}
```

设置机器状态寄存器（mstatus）中的第3位，即机器中断使能（MIE）位。用于全局启用和禁用中断的简单辅助函数如代码示例e9.15所示。关于mstatus和mie结构的完整详细信息，请参见SiFive FE310-G002 Manual中的表17和表20。

代码示例 e9.15 全局启用或禁用中断

```
void enable_interrupts() {    // 设置 mstatus 中的第 3 位 (MIE) 以启用机器中断    write_csr(mstatus, read_csr(mstatus) | (1 << 3)); void disable_interrupts() {    // 清除 mstatus 中的第 3 位 (MIE) 以禁用机器中断    write_csr(mstatus, read_csr(mstatus) & ~(1 << 3)); }
```

最后，我们将所有部分整合在一起，并在主函数中调用我们构建的函数，如代码示例 e9.16 所示。由于我们的应用程序是中断驱动的，因此在主 while 循环中我们不做任何操作。

在开发涉及安全或时序关键的应用时，使用中断需格外谨慎，因为中断是异步事件，可能在程序执行的任何时刻被触发。作为程序员，您应考虑中断在不恰当时间触发可能引入的缺陷。若有一段代码需要避免被中断，可禁用中断（即清除MIE位）。

代码示例 e9.16 使用定时器中断闪烁LED

```
#include "EasyREDVIO.h"

// CLINT memory map pointers
#define MTIMECMP ((uint64_t *) 0x02004000UL)
#define MTIME ((uint64_t *) 0x0200BFF8UL)

#define BLINK_FREQ 4 // This is an arbitrary constant used to specify the LED blink
frequency

int main(void) {

    // Set LED pin as an output
    pinMode(D13, OUTPUT);

    // Register interrupt handler.
    // The machine timer interrupt is exception code 7 as shown in so we put the
    // timer_handler() function at index 7 of the array.
    interrupt_handler[7] = timer_handler;

    // Set up interrupt by configuring mtvec
    register_trap_handler(handle_trap);

    // Reset timer
    reset_timer(MTIME_CLK_FREQ / (2 * BLINK_FREQ));

    // Enable timer interrupt
    write_csr(mie, read_csr(mie) | (1 << 7));

    enable_interrupts();

    while(1) {
    };

    return 0;
}
```

在执行指令时（mstatus），完成后重新启用中断（即在mstatus中设置MIE）。

9.4 其他微控制器外设

微控制器经常与外部外设进行接口交互。本节介绍多种常见示例，包括字符模式液晶显示器（LCD）、VGA显示器、蓝牙无线链路以及电机控制器。

9.4.1 字符型LCD

字符型LCD是一种小型液晶显示器，能够显示一行或几行文本。它们常用于收银机、激光打印机和传真机等设备的前面板，这些设备需要显示有限的信息。通过并行、RS-232或SPI接口，它们可以轻松与微控制器连接。Crystalfontz America销售多种字符型LCD，范围从8列×1行到40列×4行，并提供颜色、背光、3.3V或5V的选择。

操作及日光可见性。其LCD在小批量时成本可达20美元或更高，但大批量时价格降至5美元以下。

本节介绍如何将RED-V板与图e9.19所示的Crystalfontz CFAH2002A-TMI-JT 20 × 2并行LCD进行接口连接。该接口为8位并行接口，与最初由日立公司开发的行业标准HD44780 LCD控制器兼容。

图e9.20展示了LCD通过8位并行接口（LCD上的D0-D7输入）连接到RED-V板。LCD逻辑电路工作在5V电压下，但与RED-V板提供的3.3V输入兼容。LCD对比度由第二个电压（引脚3的输入，VO）设定，该电压通过电位器产生；通常在4.2至4.8V的设置下可读性最佳。LCD接收三个控制信号：RS（1表示字符，0表示指令）、R/W（1表示从显示屏读取，0表示写入）和E（高电平脉冲至少持续250ns，以便在下一个数据字节准备写入时使能LCD）。除了发送数据位外，当RS = 0时（即处于指令模式时），数据线（D0-D7）还用于设置LCD配置。在读取时，LCD端口D7返回忙标志，当LCD忙时该标志为1，当LCD准备好接收下一条指令或数据字节时为0。

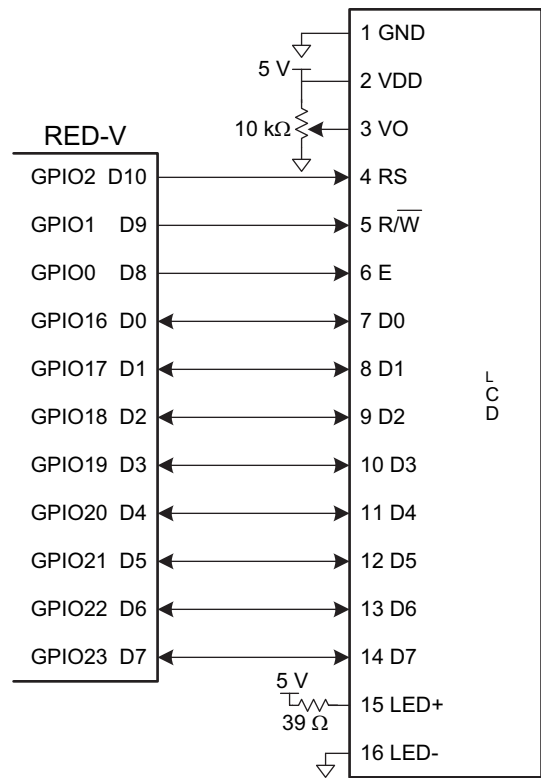
要初始化LCD，RED-V板必须按照表e9.12中的指令序列向LCD写入指令。写入指令时，将RS = 设为0，R/W = 设为0，将值置于八条数据线上，并让E脉冲至少持续250纳秒。写入数据字节时，操作相同，但需将RS = 设为1。发送指令或数据字节后，处理器必须等待至少指定时间（或有时直到忙标志清除），才能发送下一条指令或数据字节。忙标志（D7）通过将RS = 设为0、R/W = 设为1，并让E脉冲至少持续250纳秒来读取。请记住，在读取忙标志（D7）时，GPIO23也必须临时设置为输入。

配置完成后，LCD即可接收要显示的文本。通过将RS = 设为1、R/W = 设为0来向LCD写入文本，将



图e9.19 Crystalfontz CFAH2002A-TMI 20 × 2字符LCD © 2012 Crystalfontz America；经许可转载。

图 e9.20 并行 LCD 接口



八条数据线上的值，并将E脉冲保持至少250纳秒。每发送一个字符后，RED-V必须等待忙位清除才能发送下一个字符。它也可以发送指令0x01来清除显示，或发送0x02返回左上角的起始位置。

示例 e9.10 LCD 控制

编写程序，将“I love LCDs”打印到Crystalfontz CFAH2002A-TMI字符显示屏上。

解决方案 代码示例 e9.17 中的程序通过初始化显示器然后发送字符，将“I love LCDs”写入显示屏。

9.4.2 VGA显示器

一种更灵活的显示选项是驱动计算机显示器。本节将解释直接从FPGA驱动VGA (video graphics array) 显示器的底层细节。

代码示例 e9.17 向 LCD 写入 “I LOVE LCDS”

```

#include "EasyREDVIO.h"

int LCD_IO_Pins[] = {D0, D1, D2, D3, D4, D5, D6, D7};

typedef enum {INSTR, DATA} mode;
#define RS D10
#define RW D9
#define E D8

char lcdRead(mode md) {
    char c;
    pinsMode(LCD_IO_Pins, 8, INPUT);
    digitalWrite(RS, (md == DATA)); // set instr/data mode
    digitalWrite(RW, 1); // RWbar = read mode
    digitalWrite(E, 1); // pulse enable
    delay(1); // wait for LCD response
    c = digitalReads(LCD_IO_Pins, 8); // read a byte from parallel port
    digitalWrite(E, 0); // turn off enable
    delay(1);
    return c;
}

void lcdBusyWait(void) {
    char state;
    do {
        state = lcdRead(INSTR);
    } while(state & 0x80);
}

void lcdWrite(char val, mode md) {
    pinsMode(LCD_IO_Pins, 8, OUTPUT);
    digitalWrite(RS, (md == DATA)); // set instr/data mode. OUTPUT = 1, INPUT = 0
    digitalWrite(RW, 0); // set RW pin to write (RW = 0)
    digitalWrite(LCD_IO_Pins, 8, val); // write the char to the parallel port
    digitalWrite(E, 1); delay(1); // pulse E
    digitalWrite(E, 0); delay(1);
}

void lcdClear(void) {
    lcdWrite(0x01, INSTR); delay(1);
}

void lcdPrintString(char* str) {
    while (*str != 0) {
        lcdWrite(*str, DATA); lcdBusyWait();
        str++;
    }
}

void lcdInit(void) {
    pinMode(RS, OUTPUT); pinMode(RW, OUTPUT); pinMode(E, OUTPUT);
    // send initialization routine:
    delay(15);
    lcdWrite(0x30, INSTR); delay(1);
    lcdWrite(0x30, INSTR); delay(1);
    lcdWrite(0x30, INSTR); lcdBusyWait();
    lcdWrite(0x3C, INSTR); lcdBusyWait();
    lcdWrite(0x08, INSTR); lcdBusyWait();
    lcdClear();
    lcdWrite(0x06, INSTR); lcdBusyWait();
    lcdWrite(0x0C, INSTR); lcdBusyWait();
}

int main(void) {
    lcdInit();
    lcdPrintString("I love LCDS!");
}

```

表 e9.12 LCD初始化序列

Code (D7-D0)	Purpose	Wait (μs)
(apply VDD)	Allow device to turn on	15000
0x30	Set 8-bit mode	4100
0x30	Set 8-bit mode again	100
0x30	Set 8-bit mode yet again	Until busy flag is clear
0x3C	Configure 2 lines and 5 × 8 dot font	Until busy flag is clear
0x08	Turn display OFF	Until busy flag is clear
0x01	Clear display	1530
0x06	Set entry mode that increments cursor after each character	Until busy flag is clear
0x0C	Turn display ON with no cursor	

(这些是指令：所以 RS = 0 且 R/W = 0.)

VGA显示器标准于1987年针对IBM PS/2计算机推出，在 *cathode ray tube*（CRT）上提供640 × 480像素的分辨率，并通过15针连接器以模拟电压传输色彩信息。现代LCD显示器分辨率更高，但仍保持与VGA标准的向后兼容性。

在CRT中，电子枪从左到右扫描屏幕，激发荧光材料以显示图像。彩色CRT使用三种不同的荧光粉分别对应红、绿、蓝，并配备三束电子束。每束电子束的强度决定了像素中每种颜色的亮度。在每条扫描线结束时，电子枪必须关闭*horizontal blanking interval*，以便返回下一行的起始位置。当所有扫描线完成后，电子枪必须再次关闭*vertical blanking interval*，以返回左上角。整个过程每秒重复约60到75次，以刷新荧光并产生稳定图像的视觉错觉。现代显示器通常采用LCD技术，无需相同的电子扫描枪，但为了兼容性，仍使用相同的VGA接口时序。

在640 × 480像素的VGA显示器中，完整帧实际为800像素 × 525条水平扫描线，如图e9.21所示，但只有480条扫描线和每条扫描线中的640个像素真正传输图像，其余部分为黑色。一条扫描线以48像素的 *back porch* 开始，即屏幕左侧的空白区域。随后包含640个有效像素，接着是屏幕右侧16像素的空白 *front porch*，以及一个96像素时钟的水平同步（hsync）脉冲，用于快速移动。

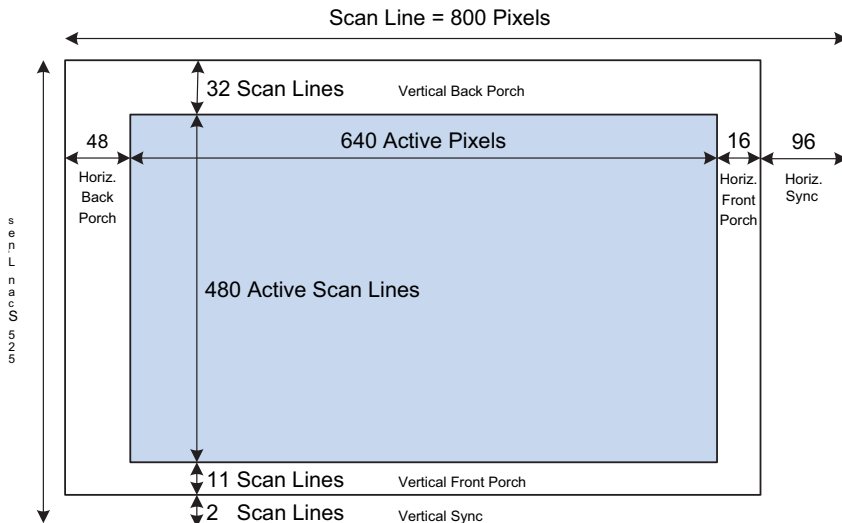


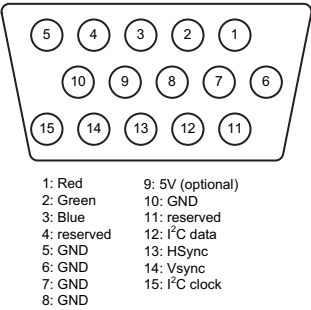
Figure e9.21 VGA frame

将枪口移回左边缘。在垂直方向上，屏幕顶部以32条扫描线的后沿开始，随后是480条有效扫描线，底部有11条扫描线的前沿，以及一个2条扫描线的垂直同步脉冲，用于返回顶部开始下一帧。对于以59.52 Hz刷新率工作的640 × 480像素VGA显示器，像素时钟以 $800 \times 525 \times 59.52 = 25 \text{ MHz}$ 运行，因此每个像素宽度为40纳秒。

图e9.22(a)显示了每条扫描线的时序。整个扫描线长度为32 μ秒。图e9.22(b)显示了垂直时序；注意此时时间单位是扫描线而非像素时钟。每秒大约绘制60帧新画面。更高分辨率使用更快的像素时钟，对于2048 × 1536分辨率以85 Hz刷新时，像素时钟高达388 MHz。例如，1024 × 768分辨率以60 Hz刷新时，可使用65 MHz像素时钟实现。

图e9.23展示了来自视频源的母头连接器的引脚排列。像素信息通过红、绿、蓝三路模拟电压传输。每路电压范围为0至0.7V，电压越高表示亮度越大。前肩和后肩期间电压应为0。视频信号需实时高速生成，这在微控制器上难以实现，但在FPGA上则较为容易。通过将数字输出引脚连接分压器，使三个颜色引脚分别输出0或0.7V，即可生成简单的黑白显示。而彩色显示器则使用video DAC，通过三个独立的D/A转换器分别驱动三个颜色引脚。

图e9.24显示了一个FPGA通过ADV7125三路8位视频DAC驱动VGA显示器。该DAC从FPGA接收8位的R、G和B信号，同时还接收一个SYNC_b信号，该信号在HSYNC或VSYNC有效时被驱动为低电平。视频DAC



图e9.23 VGA连接器引脚定义

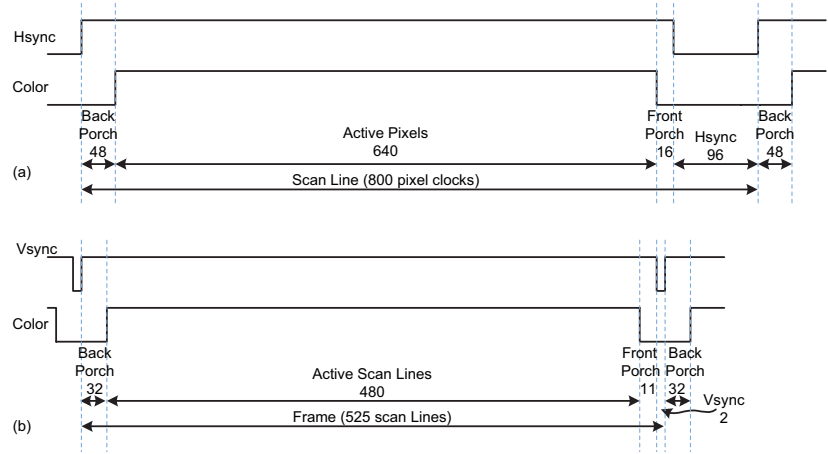
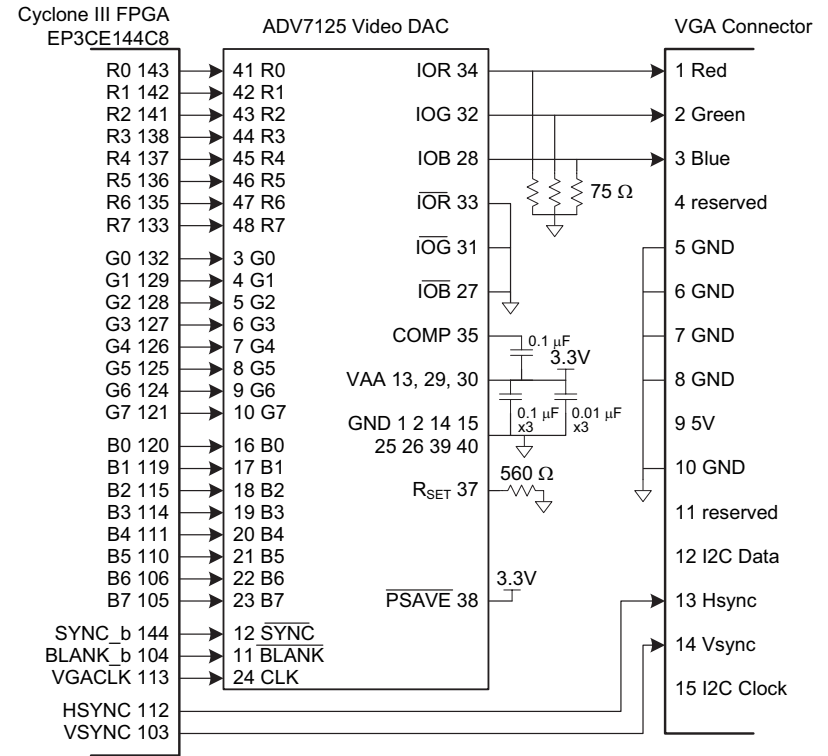


图 e9.22 VGA 时序: (a) 水平, (b) 垂直

图 e9.24 通过视频DAC驱动VGA线的FPGA



产生三路输出电流，用于驱动红色、绿色和蓝色模拟线路，这些线路通常是75 Ω传输线，在视频DAC和显示器两端均并联端接。R_{SET}电阻设置输出电流的幅度，以实现完整的色彩范围。时钟

速率取决于分辨率和刷新率；使用高速等级的ADV7125JSTZ330型号DAC时，最高可达330 MHz。

示例 e9.11 VGA 显示器显示

编写HDL代码，使用图e9.24中的电路在VGA显示器上显示文本和一个绿色方框。

解决方案 代码假设系统时钟频率为50 MHz，并使用时钟分频器生成25 MHz的VGA时钟。您也可以使用PLL生成时钟。PLL配置因FPGA而异；对于Cyclone III，频率通过Altera的兆功能向导指定。或者，VGA时钟可以直接由信号发生器提供。

VGA控制器对屏幕的列和行进行计数，在适当的时间生成hsync和vsync信号。它还会产生一个blank_b信号，当坐标超出640 × 480有效区域时，该信号被置为低电平以绘制黑色。

视频生成器根据当前像素位置 (x, y) 产生红、绿、蓝颜色值。(0, 0) 表示左上角。生成器在屏幕上绘制一组字符，以及一个绿色矩形。字符生成器绘制一个 8 × 8 像素的字符，屏幕尺寸为 80 × 60 个字符。它从 ROM 中查找字符，该字符以二进制编码为 6 列 8 行。另外两列为空白。SystemVerilog 代码反转了位顺序，因为 ROM 文件中最左侧的列是最高有效位，而它应该绘制在最低有效的 x 位置。

图 e9.25c 显示了运行此程序时 VGA 显示器的照片。字母行交替显示红色和蓝色。一个绿色框覆盖了图像的一部分。

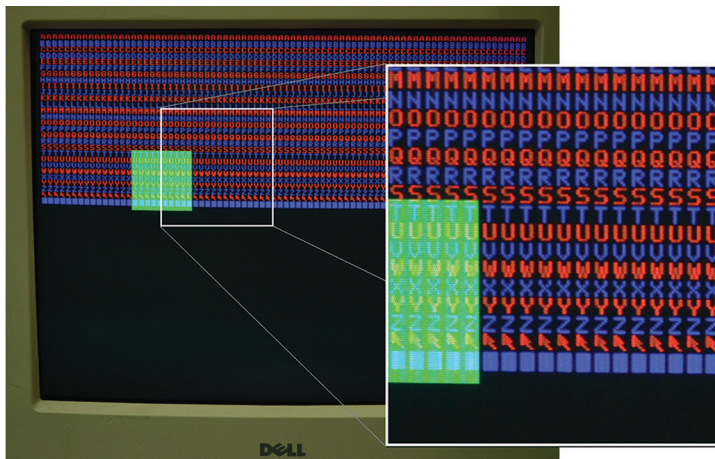


Figure e9.25 VGA output

HDL示例 e9.3 vga.sv

```

module vga(input  logic clk, reset,
           output logic vgaclk,      // 25 MHz VGA clock
           output logic hsync, vsync,
           output logic sync_b, blank_b, // to monitor & DAC
           output logic [7:0] r, g, b); // to video DAC

    logic [9:0] x, y;

    // divide 50 MHz input clock by 2 to get 25 MHz clock
    always_ff @(posedge clk, posedge reset)
        if (reset) vgaclk = 1'b0;
        else      vgaclk = ~vgaclk;

    // generate monitor timing signals
    vgaController vgaCont(vgaclk, reset, hsync, vsync, sync_b, blank_b, x, y);

    // user-defined module to determine pixel color
    videoGen videoGen(x, y, r, g, b);

endmodule

module vgaController #(parameter HBP      = 10'd48, // horizontal back porch
                          HACTIVE = 10'd640, // number of pixels per line
                          HFP      = 10'd16, // horizontal front porch
                          HSYN     = 10'd96, // horizontal sync pulse = 60 to move
                                      // electron gun back to left
                          // number of horizontal pixels (i.e., clock cycles)
                          HMAX      = HBP + HACTIVE + HFP + HSYN, //48+640+16+96=800;
                          VBP      = 10'd32, // vertical back porch
                          VACTIVE = 10'd480, // number of lines
                          VFP      = 10'd11, // vertical front porch
                          VSYN     = 10'd2,  // vertical sync pulse = 2 to move
                                      // electron gun back to top
                          // number of vertical pixels (i.e., clock cycles)
                          VMAX      = VBP + VACTIVE + VFP + VSYN) //32+480+11+2=525;

(input  logic vgaclk, reset,
 output logic hsync, vsync, sync_b, blank_b,
 output logic [9:0] hcnt, vcnt);

    // counters for horizontal and vertical positions
    always @(posedge vgaclk, posedge reset) begin
        if (reset) begin
            hcnt <= 0;
            vcnt <= 0;
        end
        else begin
            hcnt++;
            if (hcnt == HMAX) begin
                hcnt <= 0;
                vcnt++;
                if (vcnt == VMAX)
                    vcnt <= 0;
            end
        end
    end

    // compute sync signals (active low)
    assign hsync = ~( (hcnt >= (HACTIVE + HFP)) & (hcnt < (HACTIVE + HFP + HSYN)) );
    assign vsync = ~( (vcnt >= (VACTIVE + VFP)) & (vcnt < (VACTIVE + VFP + VSYN)) );
    assign sync_b = 1'b0; // this should be 0 for newer monitors
                        // for older monitors, use: assign sync_b = hsync & vsync;
    // force outputs to black when not writing pixels
    assign blank_b = (hcnt < HACTIVE) & (vcnt < VACTIVE);

endmodule

```

```

module videoGen(input logic [9:0] x, y, output logic [7:0] r, g, b);
    logic pixel, inrect;

    // given y position, choose a character to display
    // then look up the pixel value from the character ROM
    // and display it in red or blue. Also draw a green rectangle.
    chargenrom chargenromb(y[8:3]+8'd65, x[2:0], y[2:0], pixel);
    rectgen rectgen(x, y, 10'd120, 10'd150, 10'd200, 10'd230, inrect);
    assign {r, b} = (y[3]==0) ? ({8{pixel}}, 8'h00) : (8'h00, {8{pixel}});
    assign g      = inrect ? 8'hFF : 8'h00;
endmodule

module chargenrom(input logic [7:0] ch,
                  input logic [2:0] xoff, yoff,
                  output logic pixel);

    logic [5:0] charrom[2047:0]; // character generator ROM
    logic [7:0] line;           // a line read from the ROM

    // initialize ROM with characters from text file
    initial $readmemb("charrom.txt", charrom);

    // index into ROM to find line of character
    assign line = charrom[yoff+(ch-65, 3'b000)]; // subtract 65 because A
                                                // is entry 0

    // reverse order of bits
    assign pixel = line[3'd7-xoff];
endmodule

module rectgen(input logic [9:0] x, y, left, top, right, bot,
               output logic inrect);

    assign inrect = (x >= left & x < right & y >= top & y < bot);
endmodule

```

高清示例 e9.4 charrom.txt: 字符的内容

只读存储器

```

// A ASCII 65
011100
100010
100010
111110
100010
100010
100010
100010
000000
//B ASCII 66
111100
100010
100010
111100
100010
100010
111100
000000
//C ASCII 67
011100
100010
100000
100000
100000
100010
011100
000000
...

```

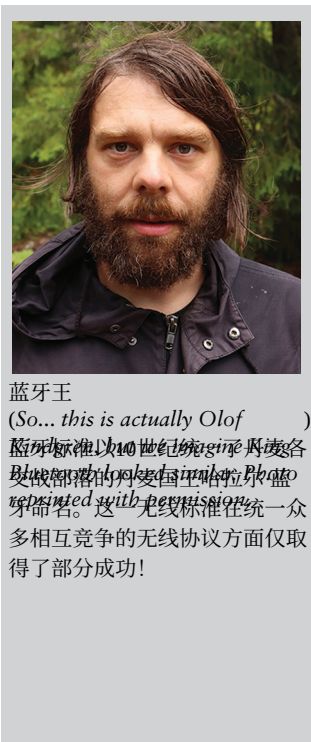


表 e9.13 蓝牙类别

Class	Transmitter Power (mW)	Range (m)
1	100	100
2	2.5	10
3	1	5

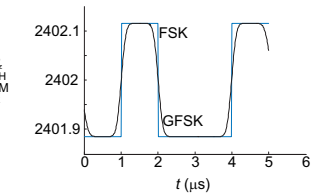


图 e9.26 FSK 和 GFSK 波形

9.4.3 蓝牙无线通信

目前已有多种无线通信标准可供使用，包括Wi-Fi、ZigBee和蓝牙。这些标准结构复杂，需要精密的集成电路，但日益丰富的模块将复杂性抽象化，为用户提供简单的无线通信接口。其中一款模块是BlueSMiRF，它是一种易于使用的蓝牙无线接口，可替代串行电缆使用。

蓝牙是一种无线标准，最初由爱立信于1994年开发，用于在5至100米距离内进行低功耗、中等速度的通信，具体距离取决于发射功率。它通常用于将耳机连接到手机或将键盘连接到电脑。与红外通信链路不同，蓝牙不需要设备之间保持直线视线。

蓝牙工作在2.4 GHz免许可工业-科学-医疗（ISM）频段。它定义了79个无线电信道，间隔为1 MHz，起始频率为2402 MHz。它通过伪随机模式在这些信道之间跳频，以避免与其他设备（例如在同一频段工作的无线路由器）产生持续干扰。如表e9.13所示，蓝牙发射器被分为三个功率等级之一，这决定了其传输距离和功耗。在基本速率模式下，它使用高斯频移键控（FSK）以1 Mbit/sec的速率工作。在普通FSK中，每个比特通过传输频率 $f_c \pm f_d$ 来传递，其中 f_c 是信道的中心频率， f_d 是至少115 kHz的偏移量。比特之间频率的突变会消耗额外带宽。在高斯FSK中，频率变化被平滑处理，以更好地利用频谱。图e9.26显示了在2402 MHz信道上使用FSK和GFSK传输一系列0和1时的频率变化。

图e9.27(a)所示的BlueSMiRF Silver模块，在一块带有串行接口的小型电路板上集成了Class 2蓝牙无线电、调制解调器及接口电路。该模块可与另一蓝牙设备（如内置蓝牙的笔记本电脑或连接PC的蓝牙USB适配器）进行通信。因此，它能在RED-V与PC之间建立无线串行链路，其功能类似于图e9.13中的有线链路，但无需使用线缆。该无线链路所兼容的软件与有线链路完全相同。

图e9.28展示了这种连接的示意图。BlueSMiRF的TX引脚连接到RED-V的RX引脚，反之亦然。RTS和CTS引脚相连，以便BlueSMiRF自行握手。

BlueSMiRF 默认设置为 115.2 kbaud，8 个数据位，1 个停止位，无奇偶校验或流控制。它采用 3.3 V 数字逻辑运行。

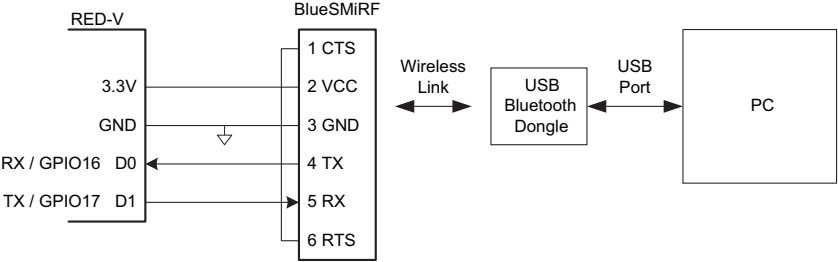


图 e9.28 BlueSMiRF RED-V 至 PC 链接

电平，因此无需RS-232收发器即可与另一个3.3V设备连接。

要使用该接口，将USB蓝牙适配器插入PC。为RED-V和BlueSMiRF上电。BlueSMiRF上的红色STAT指示灯会闪烁，表示其正在等待建立连接。打开PC系统托盘中的蓝牙图标，使用“添加蓝牙设备向导”将适配器与BlueSMiRF配对。BlueSMiRF的默认密码是1234。记下分配给适配器的COM端口号。然后即可像通过串行电缆一样进行通信。请注意，适配器通常以9600波特率运行，PuTTY必须相应配置。

9.4.4 电机控制

微控制器的另一个主要应用是驱动执行器，例如电机。本节介绍三种类型的电机：直流电机、舵机和步进电机。*DC motors*需要较高的驱动电流，通常约为1安培。因此，微控制器的GPIO无法直接驱动它们，必须在微控制器与电机之间连接一个强大的驱动器，例如*H-bridge*。如果用户想要了解电机的当前位置，电机还需要一个*shaft encoder*。

*Servo motors*接受脉冲宽度调制信号，以在有限的角度范围内指定其位置。它们非常易于接口连接，但功率较小，不适合连续旋转。

*Stepper motors*接受一系列脉冲，每个脉冲使电机旋转一个固定角度，称为*step*。它们更昂贵，并且仍然需要H桥来驱动大电流，但位置可以精确控制。

电机可能会消耗大量电流，并在电源上引入干扰，从而影响数字逻辑。减少此问题的一种方法是，为电机使用与数字逻辑不同的电源或电池。

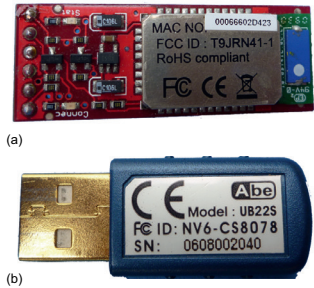


图 e9.27 (a) BlueSMiRF 模块和 (b) USB 加密狗

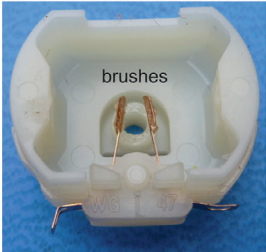
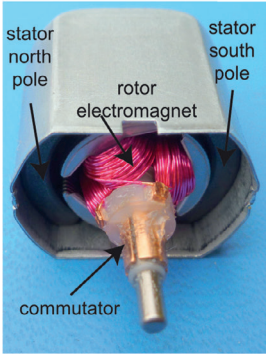
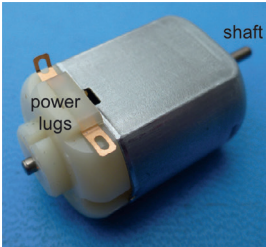


图 e9.29 直流电机

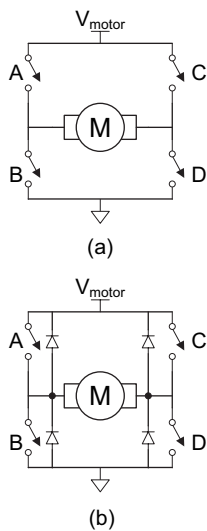


图 e9.30 H 桥

直流电机

图e9.29展示了一款有刷直流电机的结构。该电机为双端子器件，内部包含称为stator的永久固定磁铁，以及一个与转轴相连、称为rotor或armature的旋转电磁铁。转子前端连接着一个称为commutator的分体金属环。固定在电源接线片（输入端子）上的金属电刷与换向器摩擦接触，为转子的电磁铁提供电流。这会在转子中感应出磁场，使转子旋转以与定子磁场对齐。当转子旋转部分角度并接近与定子对齐时，电刷接触到换向器的相对两侧，从而反转电流方向和磁场，使转子持续旋转。

直流电机往往在极低扭矩下以每分钟数千转（RPM）的速度旋转。大多数系统会添加齿轮组，将速度降至更合理的水平并增加扭矩。请寻找专为您电机设计的齿轮组。Pittman生产多种高质量直流电机及配件，而廉价的玩具电机在爱好者中颇受欢迎。

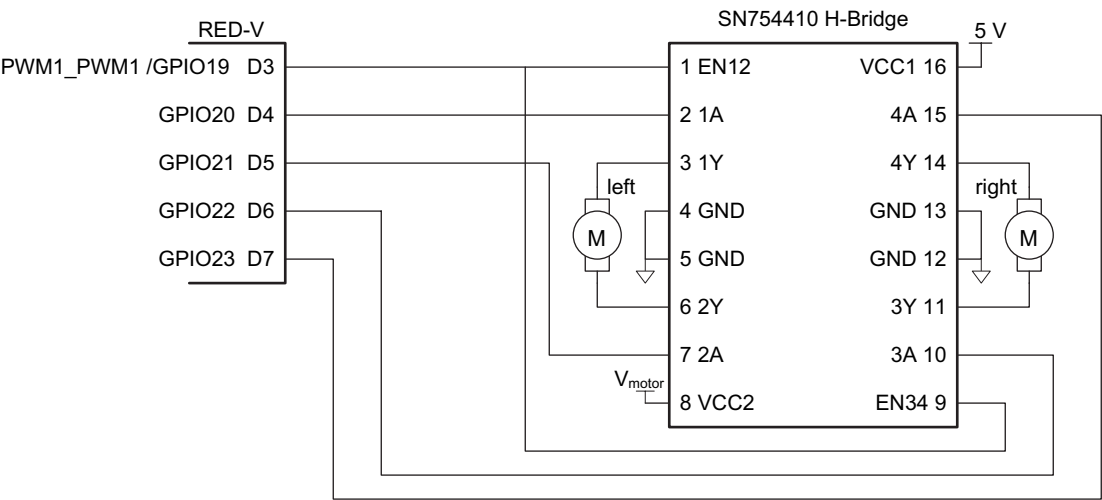
直流电机需要较大的电流和电压才能向负载提供显著功率。电流还可反向，以便电机能够双向旋转。大多数微控制器无法直接提供足够电流驱动直流电机，而是采用H桥电路。H桥在概念上包含四个电子控制开关，如图e9.30(a)所示。之所以称为H桥，是因为开关的配置形状类似于字母H。如果开关A和D闭合，电流从左向右流过电机，电机沿一个方向旋转；如果开关B和C闭合，电流从右向左流过电机，电机沿另一个方向旋转。如果A和C或B和D闭合，电机两端的电压被强制为零，导致电机主动制动。如果所有开关均断开，电机将惯性滑行至停止。H桥中的开关为{v*}，即它们能够承载一个或多个安培的大电流。H桥还包含一些数字逻辑电路，以便于控制这些开关。微控制器提供低电流数字输入来控制H桥的大电流输出。

当电机电流突变时，电机电磁铁的感应会产生较大的电压尖峰，可能损坏功率晶体管。因此，许多H桥电路还会在开关两端并联保护二极管，如图e9.30(b)所示。若感应反冲将电机任一端子驱动至 V_{motor} 以上或接地以下，二极管将导通并将电压钳制在安全水平。H桥电路会消耗大量功率，因此可能需要散热器来保持其冷却。

示例 e9.12 自主车辆

设计一个系统，使RED-V板能够控制机器人小车的两个驱动电机。编写一个函数库，用于初始化电机驱动器，并使小车能够前进、后退、左转、右转以及停止。使用PWM来改变电压输出，从而控制电机的速度。

解图e9.31展示了一对由RED-V通过德州仪器SN754410双H桥控制的直流电机。该H桥需要5V逻辑电源 V_{CC1} 和4.5至36V电机电源 V_{CC2} ；其 V_{IH} = 为2V，因此与RED-V的3.3V I/O兼容。它可为两个电机各提供高达1A的电流。 V_{motor} 应来自独立的电池组；RED-V的5V输出无法提供足够电流驱动大多数电机，且RED-V可能受损。



图e9.31 双H桥电机控制

表 e9.14 描述了每个H桥的输入如何控制电机。微控制器通过PWM信号驱动使能信号来控制电机的速度，并通过驱动其他四个引脚来控制每个电机的方向。

PWM配置为以约5 kHz的频率工作，占空比范围为0%至100%。任何远高于电机带宽的PWM频率都能实现平滑运动的效果。请注意，占空比与电机转速之间的关系是非线性的，且当占空比低于某个值时，电机将完全无法转动。

代码示例e9.18展示了如何使用图e9.31所示的双H桥配置的PWM控制来驱动两个直流电机。

表 e9.14 H桥控制

EN12	1A	2A	Motor
0	X	X	Coast
1	0	0	Brake
1	0	1	Reverse
1	1	0	Forward
1	1	1	Brake

代码示例 e9.18 直流电机驱动器

```
#include "EasyREDVIO.h"

// Motor Constants
#define EN D3
#define MOTOR_1A D4
#define MOTOR_2A D5
#define MOTOR_3A D6
#define MOTOR_4A D7

void setMotorLeft(int dir) { // dir of 1 = forward, 0 = backward
    digitalWrite(MOTOR_1A, dir);
    digitalWrite(MOTOR_2A, !dir);
}

void setMotorRight(int dir) { // dir of 1 = forward, 0 = backward
    digitalWrite(MOTOR_3A, dir);
    digitalWrite(MOTOR_4A, !dir);
}

void forward(void) {
    setMotorLeft(1); setMotorRight(1); // both motors drive forward
}

void backward(void) {
    setMotorLeft(0); setMotorRight(0); // both motors drive backward
}

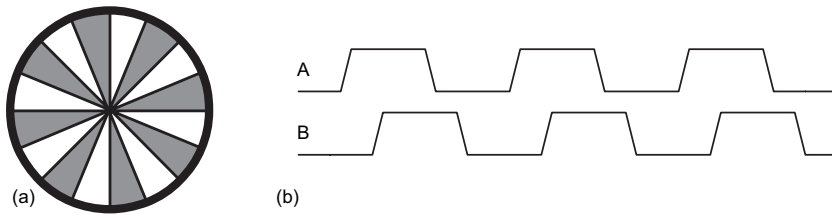
void left(void) {
    setMotorLeft(0); setMotorRight(1); // left back, right forward
}

void right(void) {
    setMotorLeft(1); setMotorRight(0); // right back, left forward
}

void halt(void) { // turn both motors off
    digitalWrite(MOTOR_1A, 0);
    digitalWrite(MOTOR_2A, 0);
    digitalWrite(MOTOR_3A, 0);
    digitalWrite(MOTOR_4A, 0);
}

void initMotors(void) {
    pinMode(MOTOR_1A, OUTPUT);
    pinMode(MOTOR_2A, OUTPUT);
    pinMode(MOTOR_3A, OUTPUT);
    pinMode(MOTOR_4A, OUTPUT);
    halt(); // ensure motors are not spinning
    pwmInit(EN, 1, 255); // turn on PWM
    analogWrite(200); // default to partial power
}

int main(void) {
    initMotors();
    while(1)
    {
        forward();
        delay(5000);
        backward();
        delay(5000);
        left();
        delay(5000);
        right();
        delay(5000);
        halt();
    }
}
```



图e9.32 轴编码器 (a) 码盘, (b) 正交输出

在前一个例子中，无法测量每个电机的位置。两个电机不太可能完全匹配，因此其中一个可能比另一个转动稍快，导致机器人偏离航向。为了解决这个问题，一些系统增加了轴编码器。图e9.32(a)展示了一个简单的轴编码器，它由一个带有槽口的圆盘组成，该圆盘安装在电机轴上。LED放置在圆盘一侧，光传感器放置在另一侧。每当槽口旋转经过LED时，轴编码器就会产生一个脉冲。微控制器可以计数这些脉冲，以测量轴转过的总角度。通过使用两个间隔半个槽口宽度的LED/传感器对，改进后的轴编码器可以产生如图e9.32(b)所示的正交输出，这些输出指示轴的旋转方向以及转过的角度。有时，轴编码器会添加另一个孔，以指示轴处于索引位置。

伺服电机

伺服电机是一种集成了齿轮组、轴编码器和部分控制逻辑的直流电机，因此更易于使用。其旋转角度有限，通常为 180° 。图e9.33展示了一个拆开盖板露出齿轮的伺服电机。伺服电机采用3引脚接口，包含电源（通常为5V）、接地和控制输入。控制输入通常为50Hz的脉宽调制信号。伺服电机的控制逻辑根据控制输入的占空比驱动转轴至相应位置。其轴编码器通常是一个旋转电位计，可产生随转轴位置变化的电压。

在典型旋转角度为 180° 的伺服电机中，1毫秒脉冲宽度驱动转轴至 0° ，1.5毫秒至 90° ，2毫秒至 180° 。例如，图e9.34展示了脉宽为1.5毫秒的控制信号。驱动伺服电机超出其范围可能导致其撞击机械限位并损坏。伺服电机的电源来自电源引脚而非控制引脚，因此控制端可直接连接微控制器而无需H桥。伺服电机因其体积小、重量轻且使用方便，常用于遥控模型飞机和小型机器人。寻找一份参数齐全的数据手册可能较为困难。

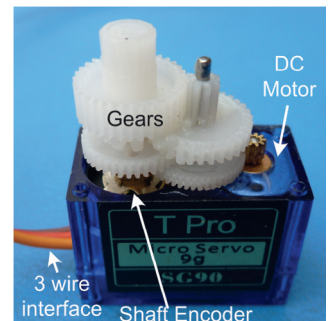


图 e9.33 SG90 舵机

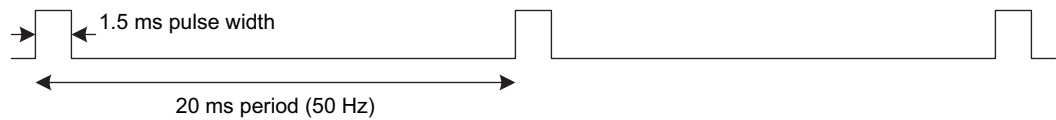


图 e9.34 伺服控制波形

困难。带有红线的中心引脚通常是电源，而黑色或棕色线通常是地线。

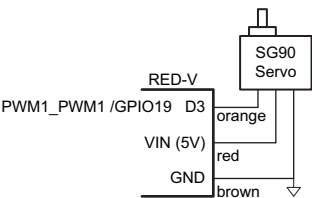


图 e9.35 伺服电机控制

示例 e9.13 伺服电机

设计一个系统，使RED-V驱动伺服电机达到所需角度。

解决方案 图e9.35展示了与SG90舵机连接的示意图，包括舵机线缆上导线的颜色。舵机的工作电压范围为4.0至7.2 V。若需输出较大力矩，其电流消耗可达0.5 A，但在负载较轻的情况下可直接由RED-V电源供电。单根导线传输PWM信号，该信号可采用5 V或3.3 V逻辑电平提供。代码配置了PWM生成方式，并计算目标角度对应的占空比。程序循环执行舵机在0°、90°和180°位置的定位。

代码示例 e9.19 伺服电机驱动器

```
#include "EasyREDVIO.h"

void genPulseMicroseconds(uint16_t pulse_len_us) {
    PWM1->pwmcmp1.pwmcmp = pulse_len_us;
}

void setServo(float angle) {
    volatile uint16_t pulse_len_us = (uint16_t) (1000 + (angle / 180) * 1000);
    genPulseMicroseconds(pulse_len_us);
}

int main(void) {
    uint32_t scale = 4; // Set scale to get 16e6/2^4 = 1 MHz count speed for 1 us accuracy
    float freq = 50.0;
    volatile uint32_t pwm_period_count = (uint32_t) (1/freq * 1e6); // Period for PWM in
                                                                    // microseconds

    pwmInit(D3, scale, pwm_period_count);
    while(1) {
        setServo(0.0);
        delay(1000);
        setServo(90.0);
        delay(1000);
        setServo(180.0);
        delay(1000);
    }
}
```

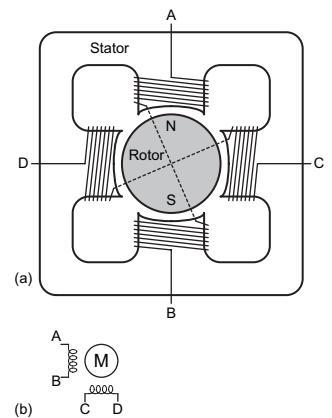
还可以通过小心拆解普通舵机、移除机械限位并用固定分压器替换电位器，将其改装为连续旋转舵机。许多网站针对特定舵机提供了详细的操作指南。改装后，PWM将控制速度而非位置：1.5毫秒表示停止，2毫秒表示全速正转，1毫秒表示全速反转。与直流电机配合H桥和齿轮组的方案相比，连续旋转舵机可能更便捷且成本更低。

步进电机

步进电机在脉冲施加到交替输入端时以离散步进方式前进。步长通常为几度，可实现精确定位和连续旋转。小型步进电机通常配备两组线圈，分别以**bipolar**或**unipolar**方式连接，称为**phases**。在相同尺寸下，双极电机功率更大且成本更低，但需要H桥驱动器；而单极电机可使用晶体管作为开关进行驱动。本节重点介绍效率更高的双极步进电机。

图e9.36(a)展示了一个简化两相双极电机，步进角为90度。转子为永磁体，具有一个北极和一个南极。定子为电磁铁，包含两对线圈，构成两个相位。因此，两相双极电机有四个端子。图e9.36(b)显示了步进电机的符号，将两个线圈建模为电感器。实际电机中会添加齿轮装置，以减小输出步进角并增加扭矩。

图e9.37展示了双相双极电机的三种常见驱动序列。图e9.37(a)展示了**wave drive**，其中线圈按AB-CD-BA-DC的顺序通电。注意，BA表示绕组AB以相反方向电流通电；这也是**bipolar**名称的由来。转子每一步转动90度。图e9.37(b)展示了**two-phase-on drive**，遵循(AB, CD)-(BA, CD)-(BA, DC)-(AB, DC)的模式。(AB, CD)表示线圈AB和CD同时通电。转子同样每一步转动90度，但会定位在两个磁极位置的正中间。这提供了最高扭矩运行，因为两个线圈同时输出功率。图e9.37(c)展示了**half-step drive**，遵循(AB, CD)-CD-(BA, CD)-BA-(BA, DC)-DC-(AB, DC)-AB的模式。转子每半个步骤转动45度。模式推进的速率决定了电机的速度。要反转电机方向，只需以相反顺序应用相同的驱动序列。



图e9.36 两相双极电机：(a) 简化示意图，(b) 符号

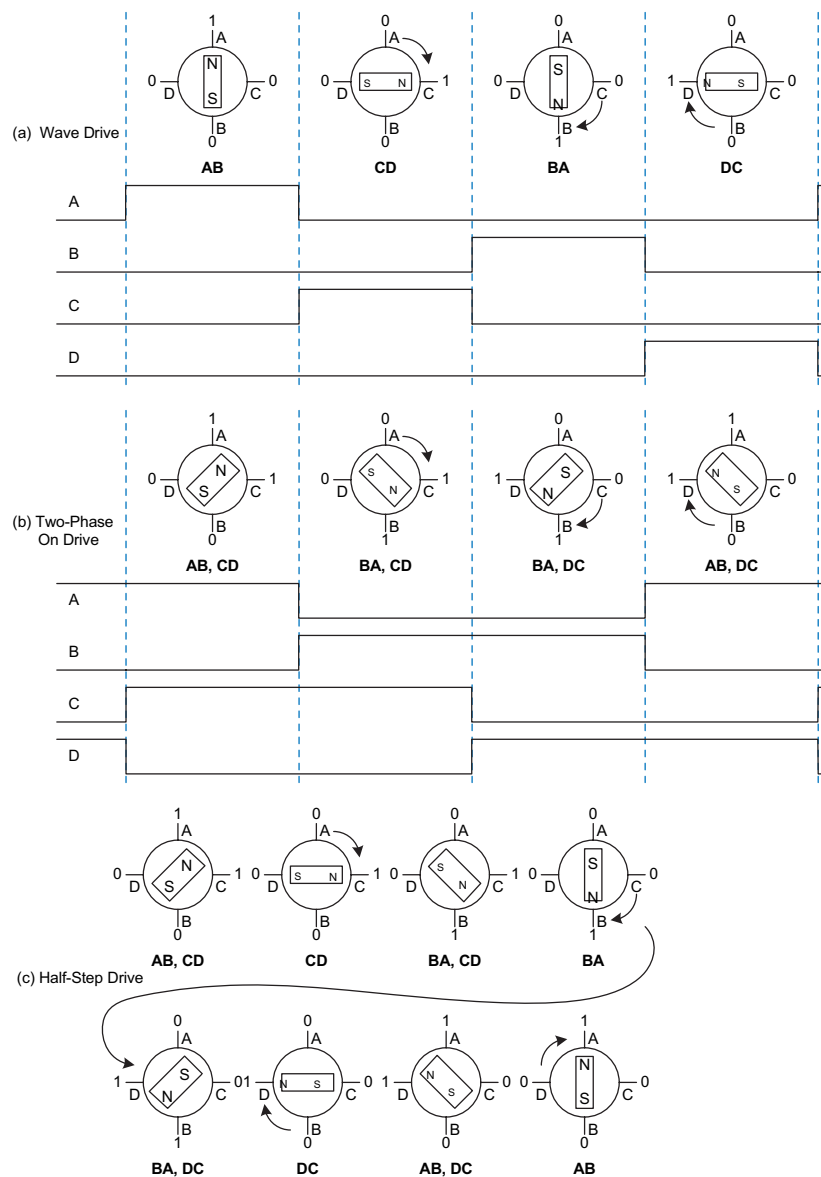


Figure e9.37 Bipolar motor drive

在实际电机中，转子具有多个磁极，以使步进角度更小。例如，图e9.39展示了一个AIRPAX LB82773-M1双极步进电机，其步进角为7.5度。该电机在5V电压下工作，每个线圈消耗0.8A电流。

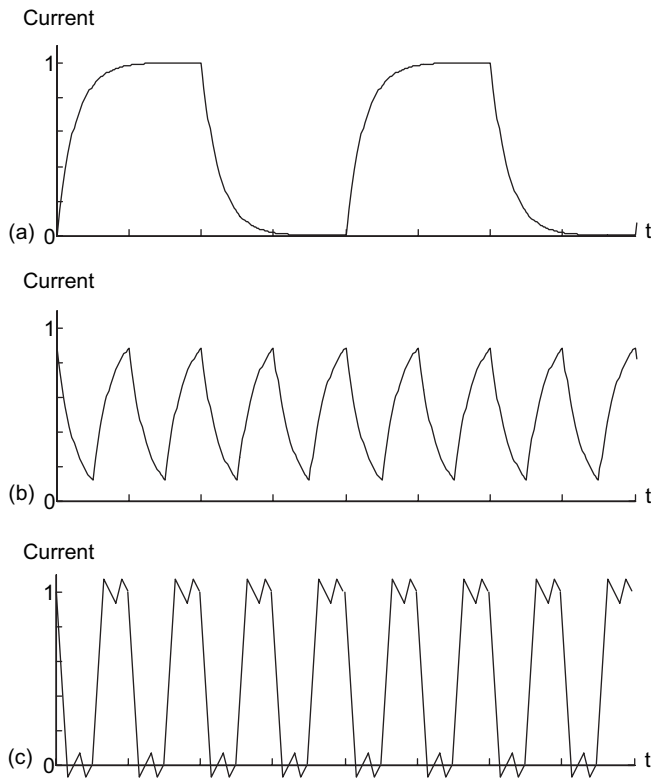
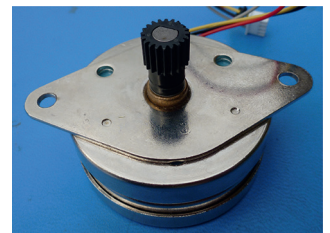


图 e9.38 双极步进电机直接驱动电流：(a) 慢速旋转，(b) 快速旋转，(c) 使用斩波驱动器快速旋转

电机中的扭矩与线圈电流成正比。该电流由施加的电压以及线圈的电感 L 和电阻 R 决定。最简单的运行模式称为*direct voltage drive*或 L/R drive，其中电压 V 直接施加到线圈上。电流以由 L/R 设定的时间常数上升至 $I=V/R$ ，如图e9.38(a)所示。这种模式适用于低速运行。然而，在较高速度下，电流没有足够的时间上升到满值，如图e9.38(b)所示，扭矩会下降。

驱动步进电机的一种更高效方式是采用脉宽调制更高电压。高电压使电流更快地升至满值；随后，在PWM占空比的关断期间切断电压，以避免电机过载。接着对电压进行调制或*chopped*，使电流维持在所需水平附近。这被称为*chopper constant current drive*，并且是



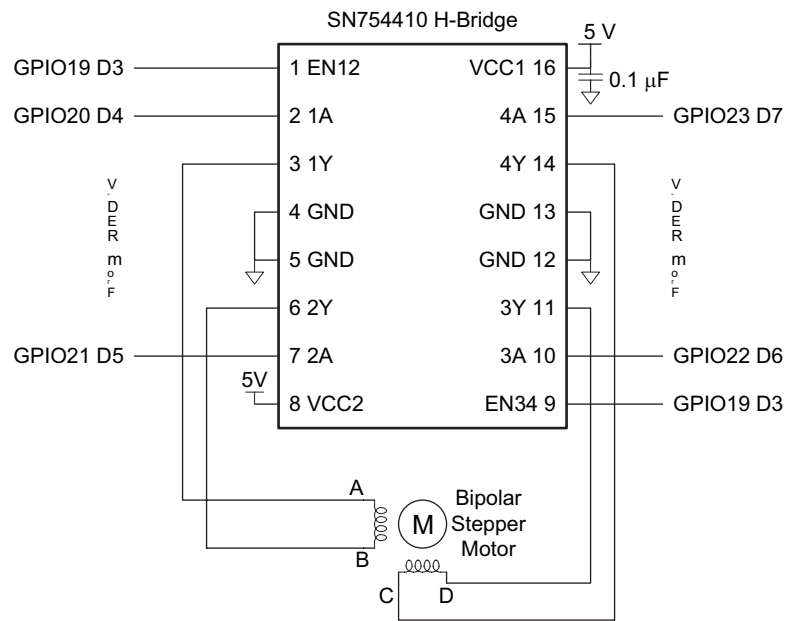
图e9.39 AIRPAX LB82773-M1双极步进电机

如图 e9.38(c) 所示。控制器使用一个与电机串联的小电阻，通过测量电压降来感知当前施加的电流，并在电流达到所需水平时向 H 桥施加使能信号以关闭驱动。原则上，微控制器可以生成正确的波形，但使用步进电机控制器更为简便。意法半导体的 L297 控制器是一个方便的选择，尤其是与 L298 双 H 桥配合使用时，后者具有电流检测引脚和 2 A 峰值功率能力。遗憾的是，L298 没有 DIP 封装，因此难以在面包板上搭建。意法半导体的应用笔记 AN460 和 AN470 是步进电机设计人员的宝贵参考资料。

示例 e9.14 双极步进电机直接波驱动

设计一个系统，使用直接波驱动方式，以指定速度和方向驱动AIRPAX双极步进电机。

解决方案 图e9.40展示了双极步进电机直接由H桥驱动，其接口与直流电机相同。请注意，VCC2必须提供足够的电压和电流以满足电机的需求，否则随着转速增加，电机可能会失步。



图e9.40 双极步进电机H桥直接驱动

Code Example e9.20 STEPPER MOTOR DRIVER

```

#include "EasyREDVIO.h"

#define STEPSIZE 7.5
#define SECS_PER_MIN 60
#define MILLIS_PER_SEC 1000
#define DEG_PER_REV 360

int stepperPins[] = {19, 22, 23, 20, 21};
int curStepState; // Keep track of the current position of stepper motor

void stepperInit(void) {
    pinsMode(stepperPins, 5, OUTPUT);
    curStepState = 0;
}

void stepperSpin(int dir, int steps, float rpm) {
    int sequence[4] = {0b00011, 0b01001, 0b00101, 0b10001}; //{2A, 1A, 4A, 3A, EN}
    int step = 0;

    unsigned int millisPerStep = (SECS_PER_MIN * MILLIS_PER_SEC * STEPSIZE) /
                                   (rpm * DEG_PER_REV);

    for (step = 0; step < steps; step++) {
        digitalWrite(stepperPins, 5, sequence[curStepState]);
        if (dir == 0) curStepState = (curStepState + 1) % 4;
        else curStepState = (curStepState + 3) % 4;
        delay(millisPerStep);
    }
}

int main(void) {
    stepperInit();
    stepperSpin(1, 12000, 120); // Spin 60 revolutions at 120 rpm
}

```

9.5 总结

大多数处理器使用内存映射I/O与现实世界进行通信。微控制器提供一系列基本外设，包括通用、串行和模拟I/O以及定时器。

本章通过SparkFun RED-V RedBoard上的FE310 RISC-V微控制器提供了许多具体的I/O示例。嵌入式系统设计者会不断遇到新的处理器和外设。集成简单嵌入式I/O的一般原则是查阅数据手册，以确定可用的外设以及涉及哪些引脚和内存映射I/O寄存器。然后，编写一个初始化外设寄存器并传输或接收数据的简单设备驱动程序通常就变得直接了当。

对于USB等更复杂的标准，编写设备驱动程序是一项高度专业化的工作，最好由对设备和USB协议栈有详细了解的专家来完成。非专业设计人员应选择一款带有经过验证的设备驱动程序和感兴趣设备示例代码的处理器。